PROJECT REPORT

CLOUD ARCHITECTURE DESIGN – AWS CLOUD PROJECT

PROJECT TITLE

Intelligent Microservice Dependency Analysis Framework for Online Shopping Platforms using Knowledge Graphs

Submitted by: Naveen Kumar (24BIT0443), Kabilesh SN (24BIT0464), [TO CONFIRM: third member name and register number]

Programme / Branch / Year: B.Tech. Information Technology, [TO CONFIRM: year]

Course Code and Name: BCSE355L – Cloud Architecture Design

Institution: Vellore Institute of Technology, Vellore

Course Instructor / Guide: Dr. Priya V

Team Number: [TO CONFIRM: team number]

GitHub Repository: https://github.com/naveenkumara2024-gif/Intelligent_Microservice_Dependency_Cloud_Project_2026-

Date of Submission: [TO CONFIRM: DD/MM/YYYY]

Abstract

Online shopping platforms built from microservices fail in cascades: when one service slows down, every service that calls it slows down too, so monitoring raises many alerts while the true origin stays hidden and recovery takes longer. Graph-based root-cause analysis addresses this by reasoning over the service dependency graph, but published methods are mostly evaluated offline on recorded data. This project builds and verifies a streaming, cloud-deployed framework that applies graph-based root-cause analysis continuously. OpenTelemetry spans from the Online Boutique demonstration shop on Amazon EKS flow through Amazon MSK to an AWS Lambda transformer that maintains a live knowledge graph of services and latency-weighted calls in Amazon Neptune. Every minute a scorer function compares each service with a learned baseline; when a service is at least twice as slow as usual, a graph neural network (GATv2 and GraphSAGE layers) on an Amazon SageMaker serverless endpoint ranks the likely root cause. Incidents are stored in Amazon DynamoDB, announced through Amazon SNS to e-mail, Slack and PagerDuty, and shown on a dashboard delivered by Amazon CloudFront and API Gateway. Because the Alibaba microservice trace dataset has no failure labels, the model was trained with synthetic fault injection on the real Alibaba call graph (7,384 services, 16,115 call edges) and then fine-tuned for small graphs. The whole system, defined as eleven AWS CDK stacks, was deployed and verified live. In a real test that added 300 ms of latency to the product-catalogue service, exactly one incident opened 1.1 minutes after the fault, the faulty service was ranked first, and the incident resolved 3.9 minutes after the fault was removed, with one alert each way. The evidence is limited to single-service latency faults; multiple simultaneous faults remain future work.

Keywords

AWS, microservices, root cause analysis, knowledge graph, graph neural network, observability

Table of Contents

Abstract 2

Keywords 2

Table of Contents 3

1. Introduction 6

1.1 Background 6

1.2 Problem Statement 6

1.3 Motivation and Significance 7

1.4 Scope 7

1.5 Report Organisation 8

2. Literature Survey / Related Work 8

2.1 Synthesis and Comparison 11

2.2 Individual Research Gap Analysis 12

2.3 Consolidated Research Gap 14

3. Project Objectives 14

3.1 Expected Outcomes 15

4. Novelty Summary 16

5. Proposed System and Architecture 17

5.1 System Overview 17

5.2 Diagram 1 – AWS Cloud Architecture (Mandatory) 18

5.3 Diagram 2 – Complete System Architecture (Mandatory) 21

5.4 Workflow / Use Cases 22

5.5 Security, Availability and Scalability Design 23

6. Dataset Description 24

6.1 Data Preparation Plan 24

7. AWS Services Planning 25
7.1 AWS Deployment and Network Design 26
7.2 Cost Estimation 27
8. Methodology and Implementation Plan 28
8.1 Overall Methodology 28
8.2 Tools and Technologies 30
8.3 Step-by-Step Implementation Plan 31
8.4 Evaluation Metrics 31
9. Implementation / Development 32
9.1 Implemented Modules 32
9.2 Screenshots and Configuration Evidence 33
9.3 GitHub Repository 42
10. Testing, Results and Observations 43
10.1 Test Plan and Test Cases 43
10.2 Results 45
10.3 Baseline Comparison 53
10.4 Results Summary 54
11. Discussion, Limitations and Risk Analysis 54
11.1 Discussion 54
11.2 Limitations 55
11.3 Risks and Mitigation 56
12. Team Contribution and GitHub Management 56
12.1 Branch and Pull Request Workflow 57
12.2 Contribution Evidence 58
13. Conclusion and Future Enhancements 58

13.1 Conclusion 58

13.2 Future Enhancements 59

References 59

Appendix A – Project Timeline 63

Appendix B – Review Checklist 64

1. Introduction

1.1 Background

Online retailers increasingly run their storefronts as microservices: separate services for the product catalogue, cart, checkout, payment, recommendations and shipping, each deployed and scaled on its own and calling the others over the network for every customer request. The style speeds up delivery, but it turns one customer action into a chain of remote calls, so the health of the platform depends on the dependencies between services as much as on any single service.

Distributed tracing, now standardised by OpenTelemetry [21], records each call as a span with its caller, callee and duration, which makes these dependencies observable. A cloud platform makes analysing them practical: managed streaming absorbs bursty trace volume, a managed graph database stores a dependency graph that changes as services are deployed, serverless functions and serverless model endpoints run only when there is work to do, and the entire system can be created and removed from code so that it costs nothing while idle.

This project observes Online Boutique [20], Google's eleven-service demonstration shop, running on Amazon Web Services (AWS) in the us-east-1 Region.

1.2 Problem Statement

When one service degrades, every service waiting on it degrades as well. A slow product catalogue makes the recommendation service and the storefront slow; per-service dashboards then raise several alarms for a single fault, and on-call engineers must correlate logs and traces across services by hand to find where the slowdown started. The symptom a customer sees is rarely where the fault originates.

The affected stakeholders are the on-call and site-reliability engineers who respond to incidents, the shoppers who experience slow or failed checkouts, and the business that loses orders while the cause is being found. Current practice falls short in three ways: threshold alerts treat services independently and therefore multiply alerts (alert fatigue); flat log and trace stores do not represent the dependency structure that explains how a fault propagates; and the graph-based root-cause methods proposed in research (Chapter 2) are mostly evaluated offline on recorded data rather than run continuously against a live system.

The problem addressed is therefore to identify continuously, from live traces, which service structurally explains a pattern of degradation, and to report it once, together with the services affected, quickly enough to shorten recovery.

1.3 Motivation and Significance

Recovery time is dominated by diagnosis: the faster responders know where a fault started, the sooner they can act. An alert that names the most likely origin and the services it affects lets them start in the right place, and one alert per incident instead of one per slow service reduces alert fatigue.

The project also shows how a research idea, a graph neural network over a service dependency graph, can be operated as a dependable cloud service: fed by live data, invoked only when needed, secured by default, observable, verified end to end, and cheap enough to run because it can be created and removed with one command each.

1.4 Scope

Included:

- Live OpenTelemetry traces from Online Boutique on Amazon EKS, streamed through Amazon MSK into a knowledge graph in Amazon Neptune.
- A graph neural network trained offline on the Alibaba microservice call graph with synthetic faults, and hosted on Amazon SageMaker.
- Minute-by-minute scoring, incident persistence in Amazon DynamoDB, alerting through Amazon SNS, and a web dashboard.
- Latency faults injected into real services, a security design with private subnets, least-privilege IAM and encryption, and infrastructure as code for repeatable deployment and teardown.

Excluded:

- Several simultaneous faults, faults in services that emit no spans (cart, shipping, ad, redis-cart, load generator), and error-rate faults (the trace data has no failure field).
- Production-scale load, multi-Region deployment, user authentication for the dashboard, and automatic remediation.

1.5 Report Organisation

Chapter 2 reviews fifteen papers and identifies the research gap. Chapter 3 states measurable objectives and Chapter 4 the novelty. Chapter 5 presents the architecture, Chapter 6 the dataset and Chapter 7 the AWS services, network design and cost. Chapter 8 describes the methodology and tools, Chapter 9 the implementation, and Chapter 10 the tests and results. Chapter 11 discusses limitations and risks, Chapter 12 the team's contributions and GitHub workflow, and Chapter 13 concludes. The appendices give the timeline and the review checklist.

2. Literature Survey / Related Work

Fifteen papers published between 2023 and 2026 were reviewed. They come from journals published by IEEE, Wiley, MDPI and other indexed venues, and from arXiv preprints, several of them accepted at ACM or IEEE conferences (FSE, ICSE-NIER, IEEE BigData). Papers are cited as P1–P15, which correspond to references [1]–[15]. Every citation was checked against the publisher's or arXiv's record in October 2026, and preprints are identified as such in the references. The papers cover tracing and trace processing (P3, P10), knowledge-graph and causal-graph diagnosis (P1, P4, P11), graph-learning anomaly detection and root-cause analysis (P2, P5, P6, P7), LLM-based root-cause analysis (P8, P9), alert correlation and availability engineering (P12, P14), and surveys (P13, P15).

Paper ID	Full citation / DOI / URL	Problem addressed	Method / architecture	Dataset / evaluation	Key findings
P1	Gu, Zhang & Du (2026). Electronics, 15(11), 2359. doi:10.3390/electronics15112359	Automated fault diagnosis with scarce labels, black-box models and costly expert rules	Unified causal graph combined with an LLM (GPT-4o) that generates diagnostic reasoning chains	Eight fault scenarios; 80/20 stratified split repeated five times	Fault matching rate 41.4 % and 33.5 % higher than fault-only and symptom-only baselines
P2	Zhang & Yang (2026). Symmetry, 18(1), 87. doi:10.3390/sym18010087	Multi-metric anomaly detection adds noise and complexity	Dynamic GNN for structure plus LSTM for time, using CPU utilisation only	Real-world microservice datasets; compared with DeepTraLog	98 % accuracy and F1 (precision and recall 0.980) against DeepTraLog F1 0.954

Paper ID	Full citation / DOI / URL	Problem addressed	Method / architecture	Dataset / evaluation	Key findings
P3	Phan et al. (2025). arXiv:2508.11342 (preprint)	Correlating spans in zero-code tracing without thread affinity or kernel changes	Greedy delay-pattern inference inside services; eBPF embeds span IDs in TCP headers across services	Not described in the abstract	Over 90 % correlation accuracy; thousands of spans correlated within seconds
P4	Wang, Qi & Wu (2024). arXiv:2402.13264 (preprint)	Slow, experience-based fault localisation over heterogeneous monitoring data	Fault-event knowledge graph from history and an online graph at failure time, compared with graph convolutional networks	Not named in the abstract; compared with state-of-the-art baselines	True root cause in the top 3 in 93.5 % of cases, within seconds
P5	Pham, Ha, Zhang & Zhang (2026). arXiv:2604.13522, accepted at FSE 2026	Services that produce no traces (blind spots) break multi-source root-cause analysis	Unsupervised: anomaly-severity scoring, clustering, causal ranking per cluster, aggregation by hypothesis testing; no call graph required	Three benchmark systems and real-world failures	Outperforms baselines when blind spots exist; real root causes appear in the top 3
P6	Zheng, Chen & Chen (2024). arXiv:2410.10021, accepted at IEEE BigData 2025	Root-cause analysis is mostly offline, or online but single-modality	OCEAN: dilated convolutions, GNN causal relations, multi-factor attention and contrastive fusion of metrics and logs	Three real-world datasets	Effective and efficient online identification (no figures in the abstract)
P7	Madabhushi et al. (2026). arXiv:2604.06448, accepted at FSE 2026 Industry	Load tests miss behaviours that appear only under live-event traffic (Prime Video)	Unsupervised GCN graph autoencoder on minute-level directed, weighted service graphs; cosine similarity of embeddings	Synthetic anomaly injection and documented incidents	About 96 % precision, 0.08 % false-positive rate, about 58 % recall

Paper ID	Full citation / DOI / URL	Problem addressed	Method / architecture	Dataset / evaluation	Key findings
P8	Zhang et al. (2026). arXiv:2601.02736, accepted at ICSE-NIER 2026	LLM-based root-cause analysis is slow and explores few hypotheses	SpecRCA: a drafting module proposes candidate root causes and a parallel verifier checks them	AIOps 2022 dataset (preliminary)	Better accuracy and efficiency than existing approaches (no figures in the abstract)
P9	Zhang et al. (2026). arXiv:2605.14866 (preprint)	LLM context explosion and serial reasoning in root-cause localisation	RCLAgent: one agent per span, organised recursively and in parallel along the trace graph	Multiple public benchmarks	Outperforms the state of the art in accuracy and inference efficiency (no figures in the abstract)
P10	Colarusso et al. (2024). Software: Practice and Experience, 54(6), 1118–1140. doi:10.1002/spe.3317	Communication raises response times; static placement cannot follow workload-dependent call graphs	Big-data pipeline that collects tracing data at run time and groups microservices for locality-aware deployment	Different working conditions and infrastructure configurations	The groupings keep the response times of critical operations low
P11	Sui et al. (2023). IEEE Transactions on Services Computing, 16(5), 3493–3507. doi:10.1109/TSC.2023.3293890	Log-based diagnosis ignores relations between log fields	LogKG: knowledge graph of log entities and relations, failure-oriented representation, OPTICS clustering, diagnosis model	A real-world and a public log dataset; deployed at a large ISP	Outperforms existing log-based diagnosis methods
P12	Behera et al. (2023). Computación y Sistemas, 27(3), 791–800. doi:10.13053/cys-27-3-4498	One failure raises alerts across many teams; recovery is slow	trACE: correlates alerts and anomalies from all subsystems back to the root cause	Microservice benchmark on AWS with a Kubernetes cluster	Mean average precision 98 % (1–34 % above baselines); 1.18 s on a dataset, 4.47 s end to end

Paper ID	Full citation / DOI / URL	Problem addressed	Method / architecture	Dataset / evaluation	Key findings
P13	Dilini, Sun, Miao & Moustafa (2026). Applied Sciences, 16(4), 1906. doi:10.3390/app16041906	Conventional anomaly-based intrusion detection: false positives, imbalance, scale	Review of 60 graph-based anomaly-detection papers (2019–2025) and a phased methodology	Survey of methods and datasets	53 % two-stage and 47 % end-to-end methods; GNNs in 18 of 28 end-to-end methods
P14	Khan et al. (2024). Eastasouth Journal of Information System and Computer Science, 2(02), 265–274. doi:10.58812/esiscs.v2i02.1077	Availability and reliability of distributed systems	Compares monolithic, microservice, load-balanced and Kubernetes-orchestrated designs	Availability, MTTR, MTTF, response time, throughput and failure rate under dynamic load	Microservices with Kubernetes give the highest availability and the shortest recovery
P15	Wang & Qi (2024). arXiv:2408.00803 (preprint)	Root-cause analysis in dense, interdependent microservice systems	Survey of metric-, trace-, log- and multi-modal methods	None (survey)	Taxonomy of methods, open challenges and future directions

2.1 Synthesis and Comparison

Approaches. Graph-based methods dominate recent work. Knowledge or causal graphs encode dependencies explicitly (P1, P4, P11), while graph neural networks learn from the structure of the service graph (P2, P6, P7); the survey in P13 counts GNNs in 18 of 28 end-to-end graph anomaly detectors. A newer line of work asks large language models to reason over traces (P1, P8, P9) and trades higher accuracy for inference cost and latency, which P8 and P9 themselves try to reduce.

Data and evaluation. Most methods are evaluated offline on recorded benchmarks or datasets (P2, P4, P5, P6, P8, P9). Only P7 (Prime Video), P11 (an internet service provider) and P12 (a benchmark on AWS with Kubernetes) report running against real or cloud-hosted systems. Their metrics differ (accuracy, top-k, mean average precision, precision and recall) and so do their datasets, so headline numbers cannot be compared directly across papers.

Cloud services, cost and security. Few papers discuss how a method is deployed. P10 builds a big-data tracing pipeline, but for service placement; P14 shows that orchestration improves

availability but leaves diagnosis to operators; none of the root-cause papers describes how its model is hosted, secured, monitored or paid for.

Trends and conflicts. The trends are towards multi-source data (P5, P6), online operation (P6), tolerance of missing telemetry (P5) and lighter models (P2). The papers disagree on two points: single-metric simplicity (P2) against multi-modal fusion (P6), and LLM reasoning (P8, P9) against learned graph models (P4, P7) when cost and latency matter. Common limitations are offline evaluation, heavy computation, dependence on historical fault records or expert-built graphs, and detection without localisation (P7, P13).

2.2 Individual Research Gap Analysis

Each student analysed the papers assigned to them in Phase I: Student A papers P1–P5 (telemetry and tracing), Student B papers P6–P10 (knowledge graphs in cloud operations) and Student C papers P11–P15 (AI-driven anomaly detection). The table records each paper's method, advantages and limitations and the gap and improvement identified by the student. The rows were checked against the papers in October 2026 and corrected where the Phase-I notes did not match the paper. [TO CONFIRM: the names of Students A, B and C, and each student's review of their rows.]

Paper ID	Existing method	Advantages	Limitations	Research gap identified by student	Possible improvement
P1 (A)	Causal graph + LLM reasoning	Automated, interpretable diagnostic chain	Graph built from expert dependencies and statistics; eight controlled scenarios	The dependency graph should be discovered from live data, not authored by experts	Derive the graph continuously from traces
P2 (A)	Dynamic GNN + LSTM on CPU only	Single metric, lightweight, 98 % accuracy	Ignores the latency signals that carry cascading slowdowns	No use of per-call latency and volume on the call graph	Use per-edge latency and call volume from traces as graph features
P3 (A)	eBPF span correlation	No code changes; over 90 % correlation	Kernel-level complexity; yields spans, not a dependency model	Correlated spans are not turned into a queryable service graph	Stream spans into a graph store as caller-to-callee edges
P4 (A)	Fault-event knowledge graph + GCN	Handles recurring faults; top-3 93.5 %	Needs historical fault-event graphs, which can grow large	No continuously updated, latency-weighted edges	Maintain live edge weights and score per time window

Paper ID	Existing method	Advantages	Limitations	Research gap identified by student	Possible improvement
P5 (A)	Unsupervised clustering + causal analysis	Works without full traces or a call graph	Relies on anomaly-severity scores; little topological context	Call-graph structure is not exploited when available	Combine deviation scores with a graph model of the topology
P6 (B)	Online multi-modal causal learning (OCEAN)	Online, multi-modal, long-term dependencies	Heavy computation for every online update	Needs a cheap trigger so expensive inference runs only when needed	Gate model calls on deviation from a learned baseline
P7 (B)	GCN graph autoencoder (Prime Video)	Unsupervised; scales to large streaming estates; 96 % precision	Recall about 58 %; flags under-represented services	Detects anomalies but does not localise a root cause	Add supervised root-cause ranking on the service graph
P8 (B)	Hypothesize-then-verify with LLMs (SpecRCA)	Parallel verification; faster LLM diagnosis	LLM cost and latency; preliminary evaluation	No native graph-database integration for verification	Check hypotheses with graph queries over a live dependency graph
P9 (B)	Multi-agent recursion over traces (RCLAgent)	Avoids context explosion; parallel	One agent per span: costly for continuous use	Requires the full trace graph for every incident	Pre-aggregate a service graph; reserve agents for explanation
P10 (B)	Runtime tracing pipeline for placement	Dynamic, collects traces at run time	Aimed at placement, not failure diagnosis	Trace-derived graphs are not reused for diagnosis	Use the same trace graph for root-cause and blast-radius analysis
P11 (C)	Log knowledge graph + clustering (LogKG)	Captures relations between log fields; deployed at an ISP	Logs only; no trace topology	Logs are not fused with traces and topology	Link log-derived failure patterns to the service call graph
P12 (C)	Alert correlation engine (trACE)	Simple and interpretable; MAP 98 %	Depends on alerts already raised by existing monitors	No learned model of how faults propagate over the topology	Rank root causes with a learned graph model
P13 (C)	Survey of graph-based anomaly detection	Broad taxonomy; shows GNN dominance	Focused on intrusion detection, not service performance	Methods not adapted to service-dependency performance faults	Adapt GNN-based detection to service call graphs
P14 (C)	Architecture comparison for availability	Quantifies the availability gains of orchestration	Improves availability by redundancy, not by diagnosis	Recovery time still depends on finding the faulty service	Pair Kubernetes resilience with automated root-cause ranking

Paper ID	Existing method	Advantages	Limitations	Research gap identified by student	Possible improvement
P15 (C)	Survey of root-cause analysis	Tracks methods across metrics, traces and logs	Descriptive; no deployment guidance	No reference deployment on a public cloud	Provide and verify a reference AWS deployment

2.3 Consolidated Research Gap

Across the fifteen papers, graph-based diagnosis is well established but almost always evaluated offline: the graph is built from a recorded dataset or historical fault events (P1, P4, P11), and models are scored on benchmarks (P2, P5, P6, P8, P9). The systems that do run continuously use their graphs for anomaly detection or placement (P7, P10), not to localise where a fault started.

What is missing is an end-to-end, continuously running system that (i) derives a latency-weighted dependency graph from live traces as they arrive (gaps noted for P1, P3 and P4); (ii) invokes graph-based root-cause ranking only when the graph shows a real deviation, so that inference stays cheap (P6, P9); (iii) turns each ranking into exactly one incident and alert with its blast radius (P12, P14); and (iv) is deployed, secured and verified on a public cloud (P15). This project addresses that gap. Its contribution is the system, not a new learning algorithm.

3. Project Objectives

The six objectives below refine the five Phase-I objectives into measurable criteria. Chapter 10 gives the evidence for each.

Objective 1 – Continuous dependency graph: build the service dependency graph continuously from live OpenTelemetry traces. Criterion: every span-emitting service appears as a caller in Neptune within minutes of deployment, and edge counters keep growing (verify:live 8/8).

Objective 2 – Managed knowledge graph: store the graph in Amazon Neptune with the same content as the local Neo4j prototype. Criterion: identical edges for the top-2,000 subset and the full 16,115-edge graph, with IAM-authenticated access only.

Objective 3 – Root-cause ranking: rank root causes with a graph neural network served by Amazon SageMaker. Criterion: on Boutique-shaped synthetic faults, top-1 accuracy of at least 60 % and top-3 of at least 90 %, and endpoint scores equal to local inference within 1e-4.

Objective 4 – Incidents and alerts: turn rankings into incidents and alerts exactly once. Criterion: no incident during a healthy soak; for a real fault, one incident within five scoring windows naming the faulty service in the top 3, one "opened" and one "resolved" alert, and an empty dead-letter queue.

Objective 5 – Visualisation: show the live topology and incidents on a web dashboard. Criterion: a slowdown is visible within one scoring window, and the dashboard verification passes through CloudFront.

Objective 6 – Secure, cost-controlled operation: keep the data tier private, apply least privilege and encryption, and deploy and remove everything with one command. Criterion: the full system deploys in under 50 minutes, and teardown leaves no billable resource (verify:teardown).

3.1 Expected Outcomes

The expected deliverables, and how each objective is assessed, are:

- A running streaming pipeline and knowledge graph (Objectives 1 and 2), assessed by verify:live and verify:neptune.
- A trained, gated model artefact and a serverless endpoint (Objective 3), assessed by the gate evaluation and verify:sagemaker.
- A scorer, incident store and alerting chain (Objective 4), assessed by verify:incidents, verify:alerts and a live fault test (verify:demo).
- A dashboard and read-only API (Objective 5), assessed by verify:dashboard.
- Eleven AWS CDK stacks with one-command deployment and teardown (Objective 6), assessed by measured deployment time and verify:teardown.
- This report, the GitHub repository and the evidence captured on 15 October 2026.

4. Novelty Summary

The table separates what this project adds from capabilities already reported in the literature reviewed in Chapter 2.

Novelty dimension	Existing approach / limitation	Proposed difference	How it will be demonstrated
New feature	Root-cause methods are evaluated on recorded traces or historical fault graphs (P4, P6, P8, P9)	A streaming trace-to-graph pipeline keeps a live, latency-weighted dependency graph in Neptune	verify:live: 14 live edges with growing counters; live topology on the dashboard
Algorithm / processing	Models score every snapshot, use a single metric (P2) or detect without localising (P7)	Per-minute window deltas against learned per-service baselines; the GNN is called only when a service is at least 2x its baseline; incident rules open after 2 windows and resolve after 3	8-minute healthy soak with 0 model calls and 0 incidents; real fault ranked #1
Cloud architecture / AWS integration	Pipelines are described conceptually or cover one stage (P10, P12)	Eleven CDK stacks: EKS, MSK, Lambda, Neptune, SageMaker, DynamoDB, SNS, API Gateway and CloudFront	All verification scripts pass live; incident 1.1 min after the fault, resolution 3.9 min after clearance
Security / automation / scalability	Manual setup; security rarely discussed	Private data subnets, one least-privilege role per function, KMS encryption, origin-secret guard; one-command deployment and teardown	Read-only access denied by KMS; full deployment in about 41 min; teardown verifier with 13 checks

The contribution of this project is a streaming, continuously updated, cloud-deployed root-cause analysis system, not a new graph-learning algorithm: graph neural networks for root-cause analysis are existing, active research (P2, P4, P6, P7). What is new relative to the reviewed work is the operational loop around them. Live traces become a latency-weighted dependency graph as they arrive; the graph is compared every minute with learned baselines; the model is consulted only when a real deviation appears; and the result becomes a single incident and alert with its blast radius. The loop is defined entirely as code and has been verified end to end on AWS with a real injected fault. No claim is made that this is the first such system; the claim is limited to what the evidence in Chapter 10 shows.

5. Proposed System and Architecture

5.1 System Overview

The users are on-call and site-reliability engineers. The input is the stream of OpenTelemetry spans that the instrumented services of Online Boutique emit for every request; a load generator inside the cluster provides steady shopping traffic. The processing steps are: collect and buffer spans; derive caller-to-callee edges and keep their counters in a knowledge graph; every minute, compare each service's latency in the last window with its learned baseline; when a service is at least twice as slow as usual, ask the graph neural network to rank the likely root causes; and open, update or resolve an incident. The outputs are incident records, alerts by e-mail, Slack and PagerDuty, and a dashboard that shows the live topology, recent latency and open incidents.

The main components are the microservices and the OpenTelemetry Collector on Amazon EKS, Kafka topics on Amazon MSK, the graph-transformer and scorer Lambda functions, the Neptune graph, the SageMaker model endpoint, the DynamoDB incident store, the SNS alerting chain and the CloudFront dashboard.

5.2 Diagram 1 – AWS Cloud Architecture (Mandatory)

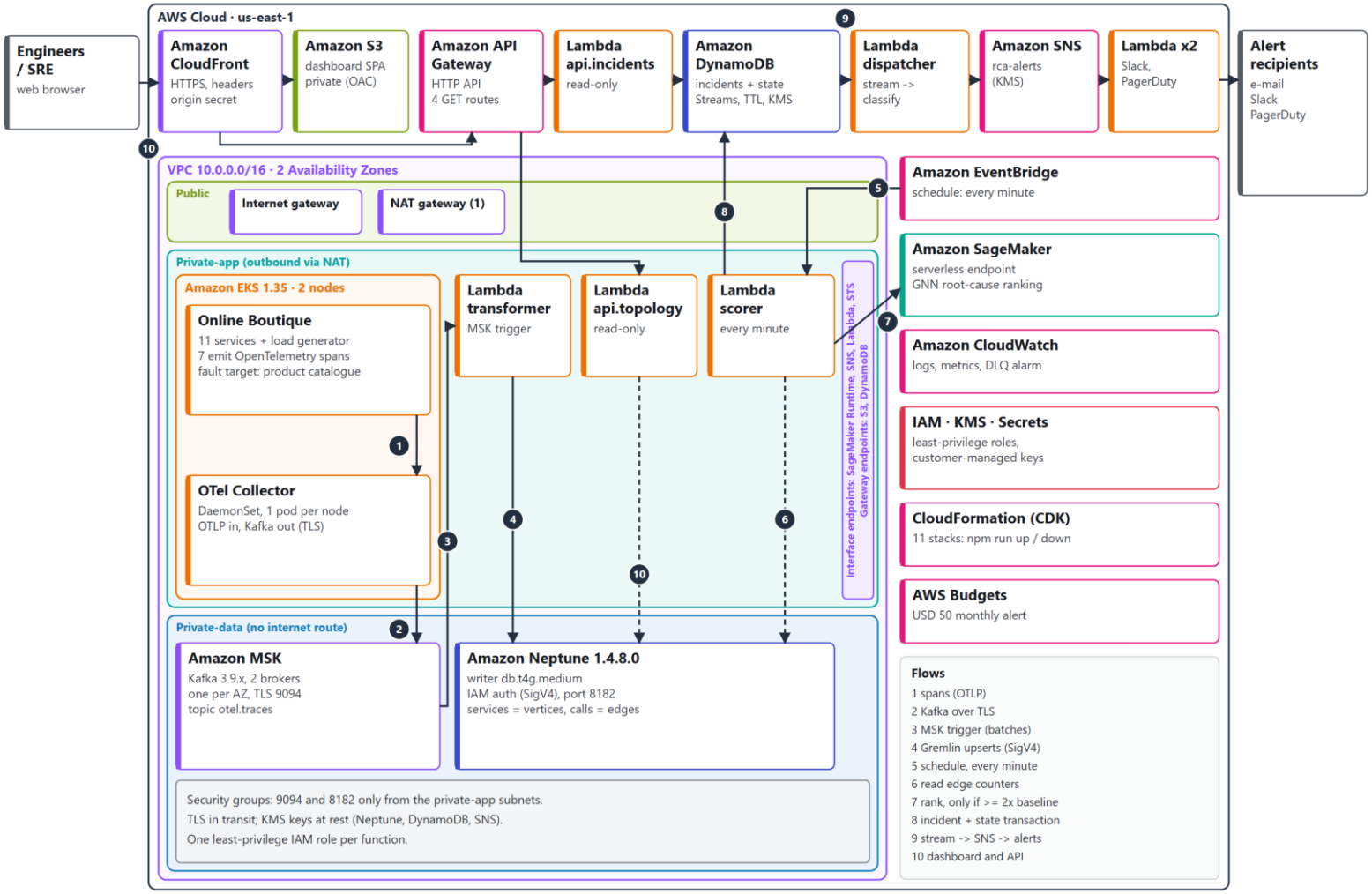


Figure 1. AWS cloud architecture of the deployed system (us-east-1). Numbered arrows are the data flows listed in the Flows box and explained below.

Figure 1 shows every AWS resource of the deployed system and the numbered data flows.

- Compute and telemetry (flows 1–2): Online Boutique v0.10.7 runs on Amazon EKS 1.35 [22] with two m7i-flex.large worker nodes, one per Availability Zone, in the private-app subnets. Seven of its services emit OpenTelemetry spans to a collector running on each node, which exports them over TLS to the Kafka topic otel.traces.
- Streaming (flow 3): Amazon MSK [23] runs Kafka 3.9.x on two kafka.t3.small brokers in the private-data subnets. An event-source mapping [24] invokes the graph-transformer Lambda function with batches of spans; its pollers reach Lambda and STS through interface endpoints because the data tier has no internet route.
- Knowledge graph (flow 4): the transformer converts each client span into a caller-to-callee edge and upserts it into Amazon Neptune 1.4.8.0 (writer db.t4g.medium) over a WebSocket signed with SigV4 (Neptune IAM database authentication [25]). Each edge keeps a call count, a latency sum, a recent average, a ten-bucket latency histogram and the time it was last seen.
- Scoring (flows 5–8): Amazon EventBridge invokes the scorer once a minute. The scorer reads the cumulative counters from Neptune, subtracts the values stored at its previous run to obtain the last window, and compares each service with its baseline. Only when a service is at least twice its baseline does it call the SageMaker serverless endpoint [26] rca-root-cause, through the SageMaker Runtime interface endpoint. Scorer state and incidents are written to DynamoDB in one transaction.
- Alerting (flow 9): the DynamoDB stream drives an alert-dispatcher function that alerts on exactly two transitions, an incident opening and an incident resolving, and publishes to the SNS topic rca-alerts. SNS delivers to the Slack and PagerDuty notifier functions and to e-mail; undeliverable messages go to an SQS dead-letter queue with a CloudWatch alarm.
- Dashboard (flow 10): CloudFront serves the single-page application from a private S3 bucket through Origin Access Control [28] and routes /api/* to an API Gateway HTTP API with four read-only routes. The api.incidents function reads DynamoDB; the api.topology function runs in the VPC and reads Neptune.
- Supporting services: CloudWatch collects logs and metrics; IAM, KMS, Secrets Manager and Systems Manager Parameter Store provide permissions, encryption keys and secrets;

CloudFormation, driven by the AWS CDK, creates and removes everything; AWS Budgets watches cost.

5.3 Diagram 2 – Complete System Architecture (Mandatory)

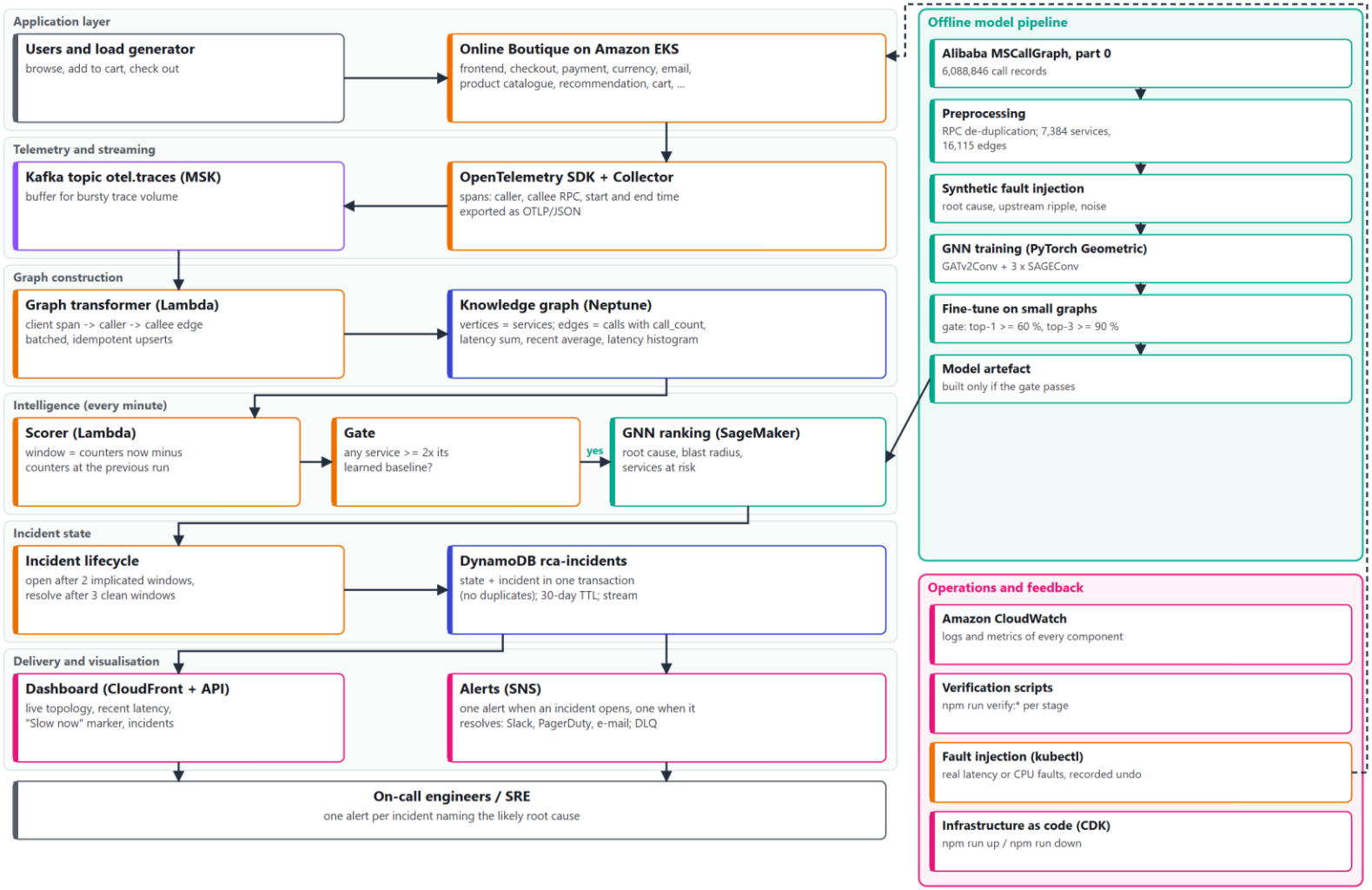


Figure 2. Complete system architecture: the continuous online path (left), the offline model pipeline that produced the deployed model, and the operations and feedback loop (right); the dashed arrow is the injection of real faults into the live shop for testing.

Figure 2 shows the complete workflow as layers. The application layer (shoppers and the load generator using Online Boutique) produces spans; the telemetry and streaming layer carries them to Kafka; the graph-construction layer turns them into a knowledge graph; the intelligence layer scores each minute and calls the model only through the 2x gate; the incident-state layer applies the lifecycle rules and stores incidents; and the delivery layer alerts engineers and updates the dashboard.

On the right, the offline model pipeline shows how the model was produced: the Alibaba call graph was preprocessed, synthetic faults were injected to create labelled examples, the GNN was trained and then fine-tuned for small graphs, and the artefact is built only when its gate evaluation passes. The operations and feedback loop closes the system: CloudWatch records every component, verification scripts check each stage, and the fault-injection tool introduces real faults into Online Boutique to test the whole path.

5.4 Workflow / Use Cases

Normal operation and the fault path proceed as follows:

1. A shopper (or the load generator) browses, adds to cart and checks out; each request becomes a tree of spans across services.
2. The collector on the node receives the spans and exports them to the otel.traces topic in MSK.
3. The graph-transformer function receives span batches, derives caller-to-callee edges (for gRPC calls, the callee is the lower-cased gRPC service name) and adds the calls and latencies to the edge counters in Neptune.
4. Each minute the scorer differences the counters, updates per-service baselines from calm windows (at least 20 calls), and checks the 2x gate.
5. If a service is at least twice its baseline, the scorer sends the window's graph to the SageMaker endpoint and receives a ranking of services by root-cause score.

6. An incident opens after two consecutive implicated windows, is updated in place while the fault lasts, and resolves after three clean windows; each opening and each resolution produces exactly one alert.

7. The engineer receives the alert naming the most likely root cause, opens the dashboard, sees the affected services and their recent latency, and starts recovery at the root cause.

5.5 Security, Availability and Scalability Design

- Network: three subnet tiers in two Availability Zones. MSK and Neptune sit in private-data subnets with no route to the internet; their security groups accept Kafka (9094) and Neptune (8182) connections only from the private-app subnets. Interface endpoints (SageMaker Runtime, SNS, Lambda, STS) and gateway endpoints (S3, DynamoDB) keep service traffic inside AWS; a reject-only VPC flow log records blocked traffic.
- Identity: every function has its own role with only the actions it needs; for example, the scorer may only read Neptune (neptune-db:ReadDataViaQuery), and only the transformer (plus a verification probe) may write to the graph. Neptune requires IAM database authentication (SigV4); unsigned requests are refused.
- Encryption: TLS in transit (Kafka, Neptune, HTTPS); customer-managed KMS keys at rest for Neptune, DynamoDB and SNS. Figure 29 shows that even a session with the ReadOnlyAccess policy cannot read incident items without permission to use the key.
- Secrets: Slack and PagerDuty credentials are SSM SecureString parameters created by the operator, never stored in code; the dashboard origin secret is generated in Secrets Manager. The dashboard API rejects requests that do not carry that secret, so the raw API Gateway URL is not a second entrance. This is a guard against direct access, not user authentication.
- Availability and fault handling: EKS nodes and MSK brokers span two Availability Zones; Neptune runs a single writer by default (a read replica in the second zone is available as an option); alert delivery is at least once with PagerDuty de-duplication by incident; failures go to a dead-letter queue with an alarm; incident records expire after 30 days through DynamoDB TTL [27].
- Scaling: Lambda functions, DynamoDB on demand and the SageMaker serverless endpoint scale with load and cost nothing when idle; MSK absorbs bursts of trace volume; the scorer calls the model only when the gate fires.

6. Dataset Description

The project uses two data sources: the Alibaba microservice trace dataset, which provides a real, large call graph for training the model, and live traces from Online Boutique, which drive the deployed system. The table describes the Alibaba dataset.

Dataset attribute	Details
Dataset name	Alibaba Cluster Trace – Microservices v2021, MS_CallGraph table [16]
Source / organisation	Alibaba Group, Alibaba Cluster Trace Program
URL	https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2021
Dataset size	MS_CallGraph is listed as 25 GiB in many part files; this project used part 0 only: 154 MiB compressed (161,892,812 bytes), 1.33 GB as CSV
Number of records / samples	6,088,846 call records in part 0; aggregated into 7,384 services and 16,115 caller-to-callee edges
Number of features	Eight columns: traceid, timestamp, rpcid, um (caller), rpctype, dm (callee), interface, rt (response time), plus a row index; there is no error or status column
Data types / format	CSV; anonymised hash identifiers, integer timestamps and response times, hierarchical rpcid strings; rpctype is one of rpc, db, mc, http, mq, userDefined
License / usage restrictions	No licence file is published; the providers ask users to cite their paper [16]. Used for academic research only.
Purpose in this project	Real service topology, call volumes and latencies for training and evaluating the root-cause model with synthetic faults
Preprocessing required	Drop rows with missing or invalid service names (NAN, "(?)", empty) and report counts; de-duplicate RPC calls recorded twice by caller and callee, keeping one call per (traceid, rpcid) with |rt|; aggregate per (caller, callee): call count, average and p95 latency
Privacy / ethical considerations	Service names are anonymised hashes; no personal or customer data. Online Boutique traffic is synthetic. Raw and processed data are kept out of the repository (gitignored) and can be regenerated.

6.1 Data Preparation Plan

Because the dataset has no failure labels, labelled examples were generated by synthetic fault injection on the real call graph. For each scenario a root-cause service with at least one caller is chosen; its impact is propagated up to four hops back through its callers, weighted by how much of each caller's traffic goes to the faulty path, and converted into relative latency deviations. Two to six unrelated distractor spikes and background jitter on every service are added so that the model cannot rely on a trivially clean signal.

For the base model, 3,000 scenarios were generated (fixed seed) and split by scenario into 70 % training, 15 % validation and 15 % test sets before training, so no scenario appears in more than one split; training ran for up to 60 epochs with early stopping on validation loss. For the deployed model, a short fine-tuning run used small random microservice topologies (8–30 services) and 8–30-service ego subgraphs sampled from the Alibaba graph. The Online Boutique topology was held out of fine-tuning and used only for evaluation, so the reported Boutique accuracy measures generalisation to an unseen graph.

7. AWS Services Planning

Only services that the project uses are listed. "Deployed" means created by the CDK stacks and checked by the named verification run.

AWS service	Purpose in this project	Input/output or integration	Status	Security / cost considerations
Amazon VPC (subnets, IGW, one NAT gateway, endpoints)	Network isolation in three tiers across two Availability Zones	Hosts EKS, MSK, Neptune and in-VPC functions; endpoints carry traffic to SageMaker, SNS, Lambda, STS, S3, DynamoDB	Deployed (verify:vpc; Figures 6, 7)	No internet route in the data tier; NAT and interface endpoints bill per hour
Amazon EKS	Runs Online Boutique and the OpenTelemetry Collector	Services emit spans to the collector	Deployed (verify:eks 4/4)	Nodes in private subnets; access entries for operators; control plane and nodes bill per hour
Amazon MSK	Durable buffer for trace bursts	Collector -> otel.traces -> transformer	Deployed (verify:msk 5/5)	TLS only; access restricted by security group; brokers bill per hour
AWS Lambda (8 functions)	Graph transformer, scorer, alert dispatcher, Slack and PagerDuty notifiers, two API functions, verification probe	MSK, EventBridge, DynamoDB stream, SNS and API Gateway invoke them	Deployed (verify:transformer 5/5 and later checks)	One least-privilege role each; billed per request and duration
Amazon EventBridge	Runs the scorer every minute	Schedule -> scorer	Deployed	No idle cost
Amazon Neptune	Knowledge graph of services and calls	Written by the transformer; read by the scorer and api.topology	Deployed (verify:neptune, verify:live 8/8)	IAM authentication (SigV4), KMS encryption, private subnets; bills per instance hour

AWS service	Purpose in this project	Input/output or integration	Status	Security / cost considerations
Amazon SageMaker	Hosts the root-cause GNN as a serverless endpoint	Scorer sends a window graph and receives a ranking	Deployed (verify:sagemaker 6/6)	No idle cost; least-privilege execution role; 2,048 MB, concurrency 2
Amazon DynamoDB	Incident records and scorer state	Scorer writes in a transaction; API reads; stream drives alerts	Deployed (verify:dynamodb 13/13)	On-demand capacity; KMS key; 30-day TTL
Amazon SNS and Amazon SQS	Alert fan-out and dead-letter queue	Dispatcher -> topic -> notifiers and e-mail	Deployed (verify:alerts; live demo)	KMS-encrypted topic; DLQ with alarm; at-least-once delivery
Amazon API Gateway	Dashboard API (HTTP API, four GET routes)	CloudFront /api/* -> API -> Lambda	Deployed (verify:dashboard)	Origin-secret guard; stage throttling
Amazon S3 and Amazon CloudFront	Hosts and delivers the dashboard	Browser -> CloudFront -> S3 and API	Deployed (verify:dashboard)	Private bucket with Origin Access Control; HTTPS with security headers
Amazon CloudWatch	Logs, metrics, alarms	Every function and endpoint	Deployed	Log groups per function; DLQ alarm
AWS IAM, KMS, Secrets Manager, SSM	Permissions, encryption keys, origin secret, alert credentials	Used by every stack	Deployed	Least privilege; customer-managed keys; secrets never in code
AWS CloudFormation (via AWS CDK) and AWS Budgets	Infrastructure as code and cost guardrail	npm run up / npm run down	Deployed; budget alert configured	Eleven stacks; USD 50 monthly budget alert
Not used: Amazon RDS, Amazon Cognito, Amazon QuickSight, standalone EC2	RDS and EC2 were not needed; QuickSight was planned in Phase I and replaced by the custom dashboard; Cognito is future work	–	Not used	–

7.1 AWS Deployment and Network Design

All resources are in us-east-1. The VPC uses 10.0.0.0/16 across us-east-1a and us-east-1b with three /24 subnet tiers per zone: public (internet gateway and one NAT gateway), private-app (EKS nodes and in-VPC functions, with outbound access through NAT) and private-data (MSK and Neptune, with no route to the internet). Security groups allow Kafka and Neptune traffic

only from the private-app subnets, and the interface endpoints accept HTTPS only from inside the VPC.

The infrastructure is defined in TypeScript with the AWS CDK [29] as eleven stacks that depend on each other in a fixed order: VPC first; EKS, MSK and Neptune next; then the transformer, scorer, API and dashboard. DynamoDB, SageMaker and alerting have no VPC dependency. One command, npm run up, runs safety checks (credentials, budget alert, bootstrap), builds the code and model artefact once, deploys all stacks in parallel and applies the Kubernetes workloads; npm run down clears any injected fault, removes Kubernetes load balancers, destroys all stacks in parallel and runs a 13-check sweep that confirms nothing billable is left.

Access boundaries: operators use the AWS CLI and kubectl (through the EKS public endpoint with an access entry); users reach only CloudFront; services reach each other through security groups and VPC endpoints.

7.2 Cost Estimation

Measured cost. Cost Explorer shows USD 8.51 of usage from 1 to 15 October 2026 before credits (EC2 compute 2.96, EKS 1.65, VPC 1.35, other EC2 charges 1.34, MSK 1.15, KMS 0.04), matching the budget page in Figure 3; account credits covered it, so the net amount billed was about USD 0. These figures cover the earlier verification sessions; Cost Explorer had not yet posted the 15 October session when queried.

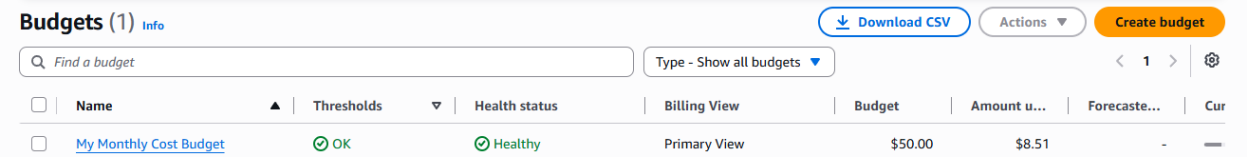


Figure 3. AWS Budgets: the USD 50 monthly budget alert and USD 8.51 of usage in October 2026 before credits.

Estimated running cost (estimate, not a bill). On-demand prices from the AWS Price List API [30] for us-east-1, retrieved on 15 October 2026, give the hourly cost of the resources that bill while the system is up:

- EKS control plane USD 0.10; two m7i-flex.large nodes 2 x 0.09576 = 0.19; two kafka.t3.small brokers 2 x 0.0456 = 0.09; Neptune db.t4g.medium 0.093; NAT gateway 0.045; four interface endpoints in two zones 8 x 0.01 = 0.08.
- Total about USD 0.60 per hour, or about USD 440 per month if left running continuously (730 hours).
- Not included: storage (MSK 10 GB per broker at USD 0.10 per GB-month, node volumes), data processed by the NAT gateway, and request-priced services (Lambda, SageMaker serverless, DynamoDB on demand, SNS, API Gateway, CloudFront), which are small at demonstration load.

Cost control. The budget alert is checked before every deployment; the system is deployed only for working sessions and removed afterwards with npm run down; serverless SageMaker, on-demand DynamoDB and Lambda cost nothing while idle; there is one NAT gateway instead of one per zone, and the Neptune read replica is off by default. A three-hour session therefore costs about USD 2 of hourly charges.

8. Methodology and Implementation Plan

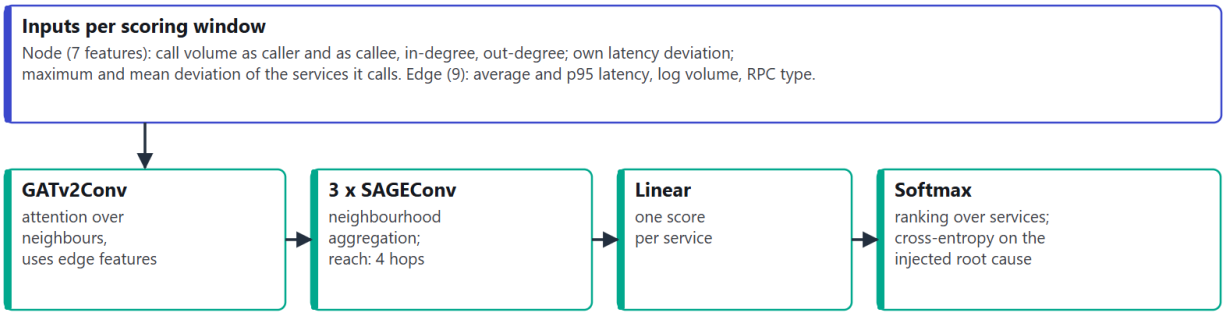
8.1 Overall Methodology

The project followed sixteen stages (0–15), each with a written plan, an explicit checkpoint and a record of what was built, kept in the prompt/ folder of the repository. Work moved from local to cloud: the dataset and graph were prototyped locally (Neo4j and Gremlin Server in Docker, DynamoDB Local), the model was trained offline, and each AWS service was added only after the previous stage's checkpoint passed.

- Requirements and literature (Phase I): problem definition, fifteen-paper survey, research gap, objectives and novelty.
- Data acquisition and local prototype (Stages 0–2): one Alibaba part file, preprocessing into nodes and edges, and a Neo4j graph.
- Model (Stages 3 and 10): GNN trained on synthetic faults, fine-tuned for small graphs, gated and served from SageMaker.

- Design and implementation (Stages 4–13): CDK stacks, then networking, EKS, MSK, the transformer, Neptune, incident persistence, alerting and the dashboard.
- Integration and evaluation (Stage 14): real faults injected into running services and judged by pre-written rules.
- Operations (Stage 15): one-command deployment and teardown with a cost sweep.

Figure 4 shows the model. Each scoring window becomes a graph whose nodes carry four static features (call volume as caller and as callee, in-degree and out-degree, scaled) and three dynamic ones (the service's own latency deviation and the maximum and mean deviation of the services it calls); edges carry nine features (average and p95 latency, log call volume and a one-hot RPC type). A GATv2 attention layer [17] followed by three GraphSAGE layers [18] gives each service information from up to four hops, and a linear layer produces one score per service; a softmax over services is trained with cross-entropy against the injected root cause. The key signal is the pattern in the table of Figure 4: a root cause is slow while the services it calls are fine, whereas the services above it are slow because something they call is slow. For serving, the layers were re-implemented in plain PyTorch, and their outputs match the PyTorch Geometric [19] model exactly on the 7,384-node graph.



Why it separates cause from symptom (calls A -> B -> C, fault at C)

Service	Own deviation	Deviation of what it calls	Interpretation
A	medium	high	slow because something it calls is slow
B	high	very high	relays the fault upstream
C	very high	**normal**	**slow while what it calls is fine: root cause**

Labels come from synthetic fault injection on the real Alibaba call graph; the trace data has no failure column.

Figure 4. Root-cause model: a GATv2 layer and three GraphSAGE layers produce one score per service; the table shows the latency pattern that distinguishes a root cause from the services that merely relay its slowdown.

8.2 Tools and Technologies

Tool / technology	Purpose	Version / configuration
AWS services	Hosting, streaming, graph storage, inference, persistence, alerting, delivery	us-east-1; EKS 1.35; MSK Kafka 3.9.x; Neptune 1.4.8.0; SageMaker PyTorch inference container 2.6.0-cpu-py312
AWS CDK and TypeScript	Infrastructure as code (eleven stacks)	aws-cdk CLI 2.1142.0; aws-cdk-lib 2.270.0; TypeScript 7.0
Node.js and TypeScript	Lambda functions (transformer, scorer, alerts, API) bundled with esbuild	Lambda runtime Node.js 22; local Node.js 24; gremlin client 3.7.3; AWS SDK v3
Python, PyTorch, PyTorch Geometric	Preprocessing, GNN training and serving	Python 3.14 (local) and 3.12 (endpoint); PyTorch 2.11 (training) and 2.6 (endpoint); PyTorch Geometric 2.8; pandas 3.0
Kubernetes and OpenTelemetry	Demo application and telemetry collection	Online Boutique v0.10.7; OpenTelemetry Collector Contrib 0.160.0; kubectl
React, Vite and Cytoscape.js	Dashboard single-page application	React 19.3; Vite 8.3; Cytoscape.js 3.34
Neo4j, Gremlin Server, DynamoDB Local (Docker)	Local stand-ins before deploying to AWS	Neo4j 5 Community; Gremlin Server 3.7.3
Testing	Unit, integration and infrastructure tests	Jest 30, Vitest 5, pytest 9, CDK assertions

Tool / technology	Purpose	Version / configuration
Git and GitHub	Version control and collaboration	https://github.com/naveenkumara2024-gif/Intelligent_Microservice_Dependency_Cloud_Project_2026- ; main branch with one branch per stage

8.3 Step-by-Step Implementation Plan

Step 1 – Requirements and success criteria: problem statement, objectives and a measurable checkpoint for every stage, written before the stage was built.

Step 2 – Dataset or input preparation: download one Alibaba part file, verify its real schema with pandas, preprocess it into 7,384 nodes and 16,115 edges, and load it into a local Neo4j graph with idempotent MERGE writes.

Step 3 – Architecture and AWS resource setup: model the system as CDK stacks, validate with cdk synth and unit tests, then deploy the VPC, EKS, MSK and Neptune and check each with its verification script.

Step 4 – Application / backend / model development: train and fine-tune the GNN; build the graph transformer, scorer, alerting functions, API and dashboard, each first against local stand-ins.

Step 5 – Integration and security configuration: connect the stages (collector to MSK, MSK trigger, SigV4 to Neptune, VPC endpoints, KMS keys, least-privilege roles), fixing the defects that only appeared on AWS.

Step 6 – Testing and performance evaluation: run the verification scripts, measure endpoint latency and scorer behaviour, and inject a real fault into a running service.

Step 7 – Deployment, monitoring and documentation: one-command deployment and teardown, CloudWatch monitoring, the evidence captured on 15 October 2026, and this report.

8.4 Evaluation Metrics

- Root-cause ranking accuracy: share of synthetic scenarios in which the injected root cause is ranked first (top-1) or among the first three (top-3), against a trivial baseline (the service with the largest own deviation) and random choice.

- Detection delay: time from a fault reaching traffic to the incident opening; resolution delay: time from clearing the fault to the incident resolving (scoring windows of one minute).
- Correctness of alerting: incidents and alerts per fault (target exactly one each way), false incidents during a healthy soak, failed deliveries and dead-letter messages.
- Endpoint latency (cold and warm), scorer run time and errors from CloudWatch, and parity between the endpoint and local inference.
- Deployment and teardown time, and cost (measured in Cost Explorer; estimated from the Price List).

9. Implementation / Development

All modules below are implemented and were verified on AWS. Screenshots were taken on 15 October 2026 during a full deployment; account identifiers, e-mail addresses and the CloudFront domain are masked, and no credentials appear in the report.

9.1 Implemented Modules

Module	Implementation details	AWS services / tools	Evidence (figure, commit, PR)	Status
Dataset preprocessing and local graph	Schema-checked preprocessing with RPC de-duplication; Neo4j prototype with MERGE	Python, pandas, Neo4j (Docker)	Stages 1–2, commit of 21 Sep 2026	Complete
GNN training and serving model	Synthetic fault injection; GATv2 + GraphSAGE; fine-tuning for small graphs; gated artefact; plain-PyTorch serving copy	PyTorch, PyTorch Geometric	Stages 3, 10; boutique_eval.json; Figure 27	Complete
Networking	VPC, three subnet tiers, endpoints, security groups, flow log	VPC, CDK	verify:vpc; Figures 6, 7	Deployed
Microservices and telemetry	Online Boutique with tracing enabled; collector DaemonSet; Kafka exporter overlay	EKS, OpenTelemetry	verify:eks 4/4; Figures 8, 9, 10	Deployed
Streaming	MSK cluster, TLS, topic-creation job	MSK	verify:msk 5/5; Figure 11	Deployed

Module	Implementation details	AWS services / tools	Evidence (figure, commit, PR)	Status
Graph transformer	OTLP parsing, client-span edge derivation (gRPC-aware), batch aggregation, SigV4-signed Gremlin writes	Lambda, Neptune	verify:transformer 5/5, verify:live 8/8; Figures 13, 14	Deployed
Knowledge graph	Neptune cluster with IAM auth; parity with the Neo4j prototype	Neptune	verify:neptune; Figure 12	Deployed
Model endpoint	Serverless endpoint; 4xx for malformed input	SageMaker	verify:sagemaker 6/6; Figure 16	Deployed
Scorer and incident store	Window deltas, EWMA baselines, 2x gate, lifecycle rules, transactional writes	Lambda, EventBridge, DynamoDB	verify:incidents 22 PASS; Figures 15, 17	Deployed
Alerting	Stream classifier; Slack, PagerDuty and e-mail delivery; DLQ	Lambda, SNS, SQS	verify:alerts 18 PASS; Figure 18	Deployed (Slack and PagerDuty credentials not yet configured)
Dashboard and API	React and Cytoscape topology, recent-window latency, incidents; read-only API	S3, CloudFront, API Gateway, Lambda	verify:dashboard 20 PASS; Figures 19, 20, 24	Deployed
Fault injection and demo judge	Real latency and CPU faults with recorded undo; pre-written pass rules	kubectl, CloudWatch	verify:demo 19/19 (14 and 15 Oct 2026); Figure 26	Complete
One-command operations	npm run up / npm run down / verify:teardown	CDK, CloudFormation	Figure 5; deployment in about 41 min	Deployment verified; teardown [TO CONFIRM after npm run down]

9.2 Screenshots and Configuration Evidence

The screenshots follow the data path from the infrastructure to the model endpoint.

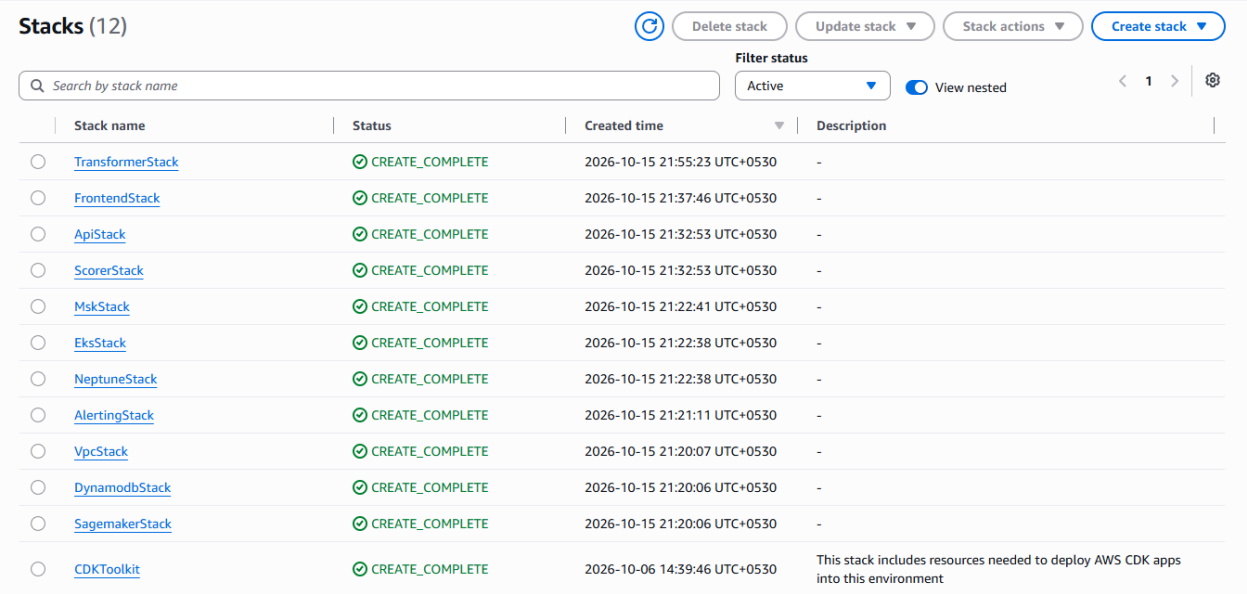


Figure 5. AWS CloudFormation: the eleven project stacks in CREATE_COMPLETE after one npm run up (15 October 2026; times in IST).

One command deployed all eleven stacks; CloudFormation records show the first stack starting at 15:50 UTC and the last finishing at 16:31 UTC, about 41 minutes for the whole system (Figure 5).

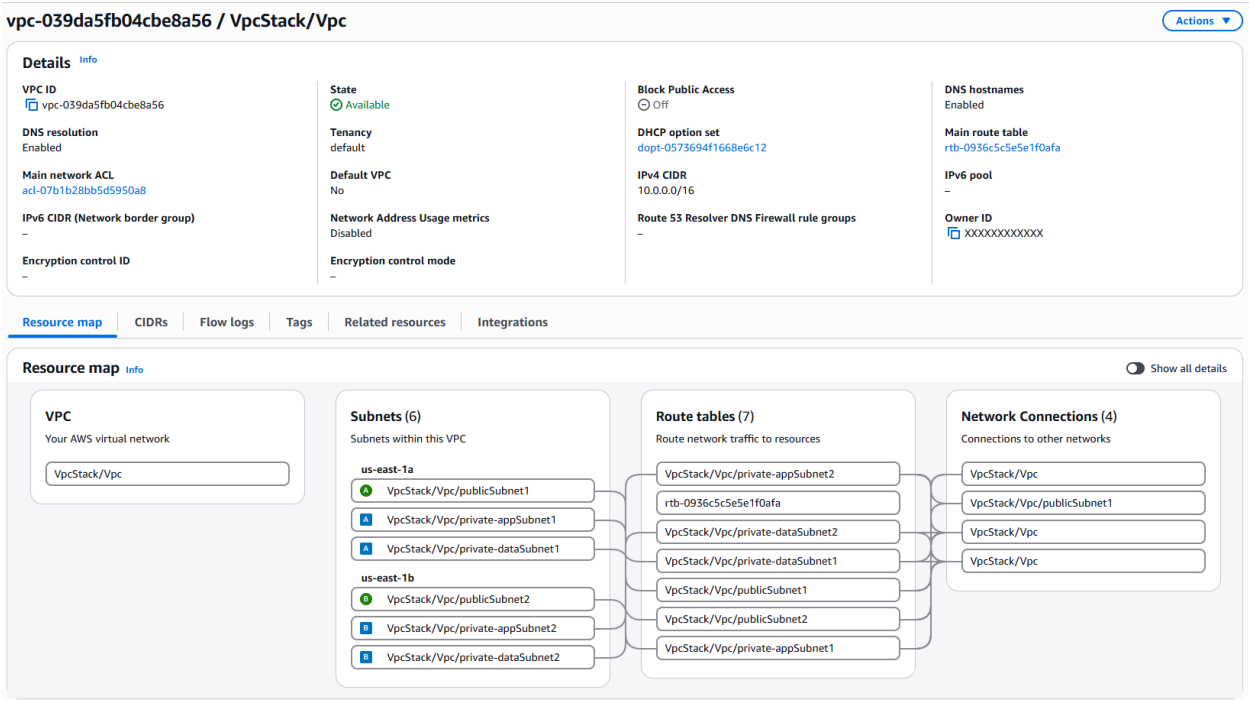


Figure 6. Amazon VPC resource map: 10.0.0.0/16 with public, private-app and private-data subnets in us-east-1a and us-east-1b (owner ID masked).

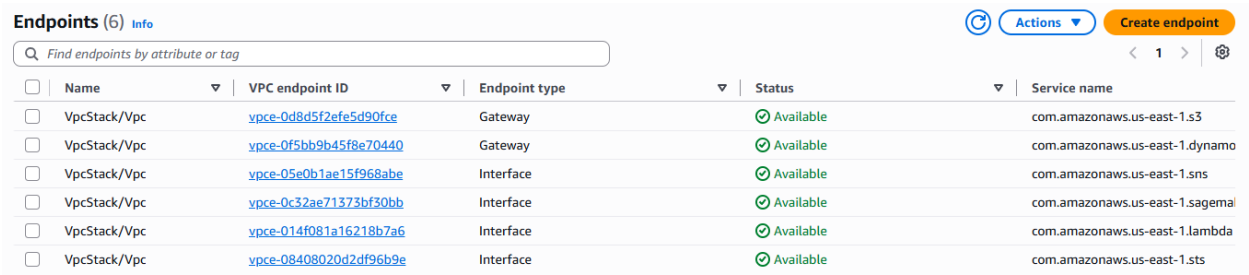


Figure 7. VPC endpoints: gateway endpoints for S3 and DynamoDB and interface endpoints for SNS, SageMaker Runtime, Lambda and STS, all Available.

Figures 6 and 7 show the network of Section 7.1: six subnets in two zones and six endpoints.

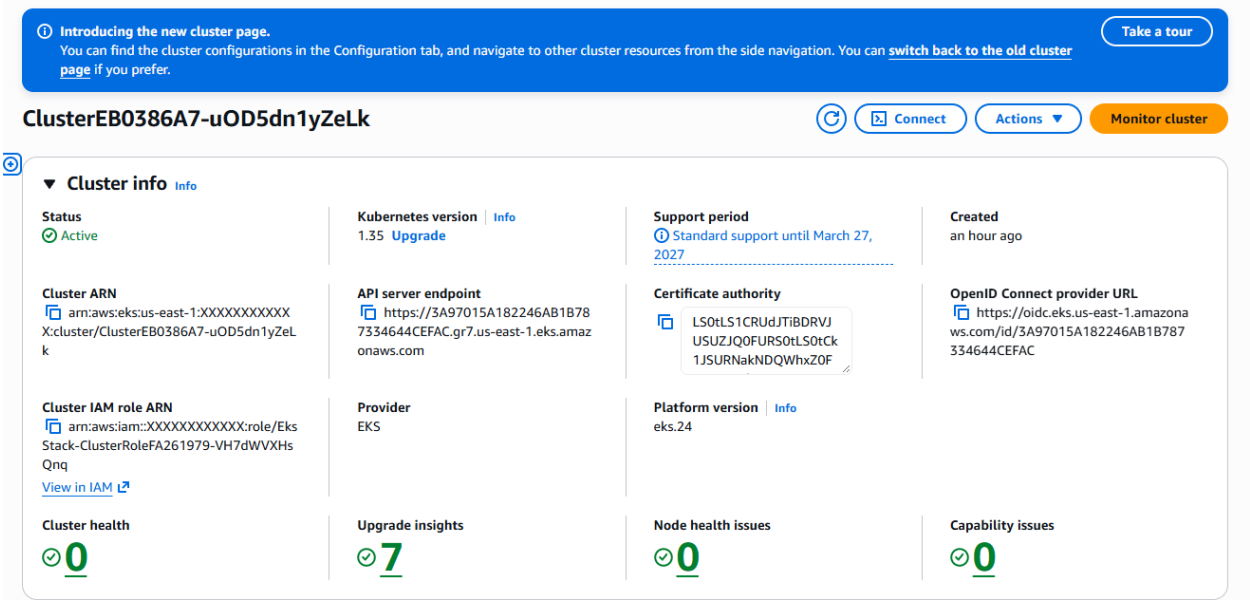


Figure 8. Amazon EKS cluster: Active, Kubernetes 1.35, no health or node issues (account ID masked).

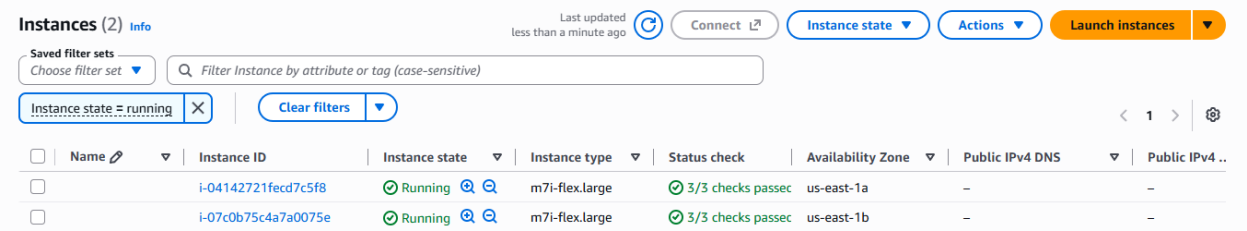


Figure 9. EKS worker nodes: two m7i-flex.large instances, one in each Availability Zone, passing all status checks.

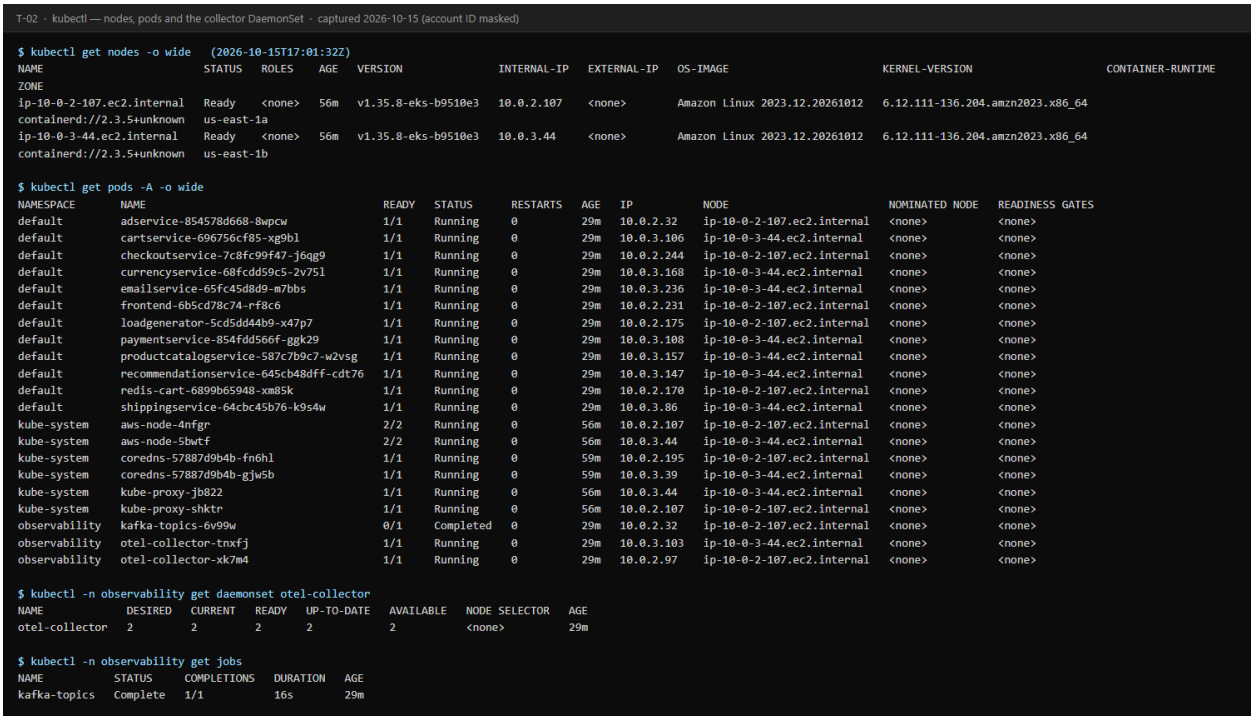


Figure 10. kubectl evidence: both nodes Ready, all twelve Online Boutique pods Running, the collector DaemonSet 2/2 and the Kafka topic job Completed.

The cluster runs the shop and the collector on two nodes in different zones (Figures 8–10). The EKS console's node and pod views need Kubernetes permissions that the read-only screenshot session deliberately lacked, so the node and pod evidence comes from the EC2 console and kubectl.

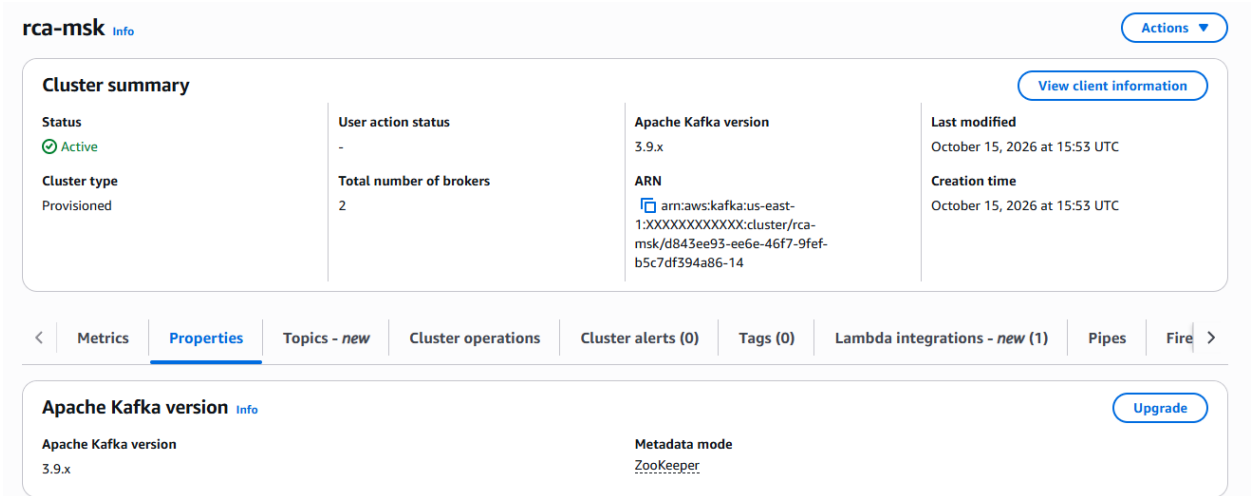


Figure 11. Amazon MSK: provisioned cluster rca-msk, Active, Apache Kafka 3.9.x, two brokers.

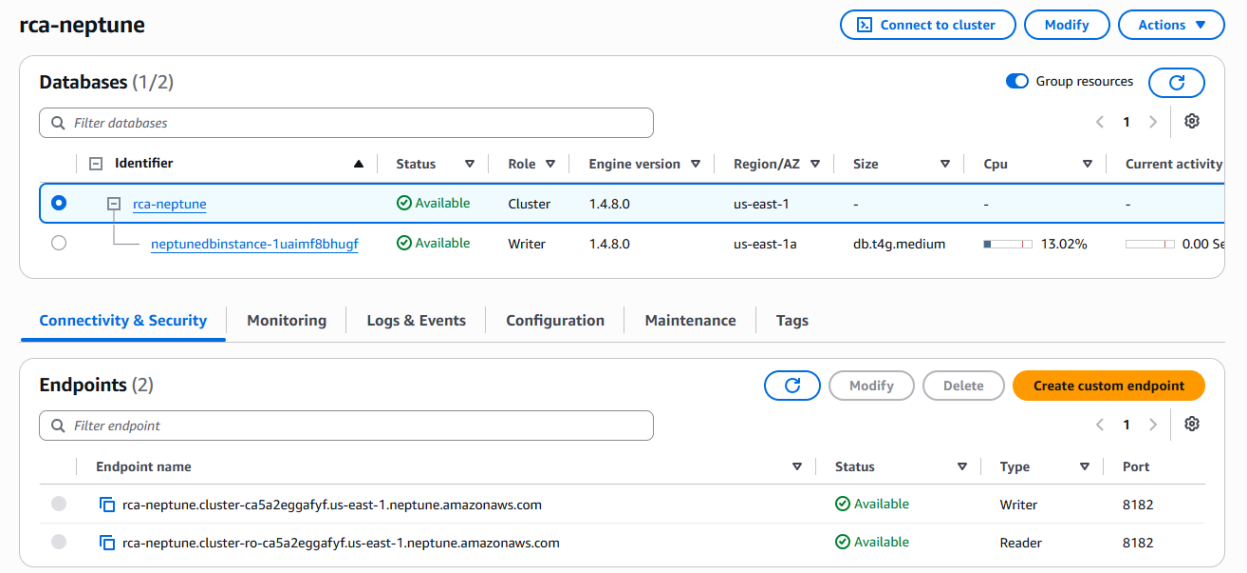


Figure 12. Amazon Neptune: cluster rca-neptune and its writer instance (db.t4g.medium, engine 1.4.8.0) Available in us-east-1a.

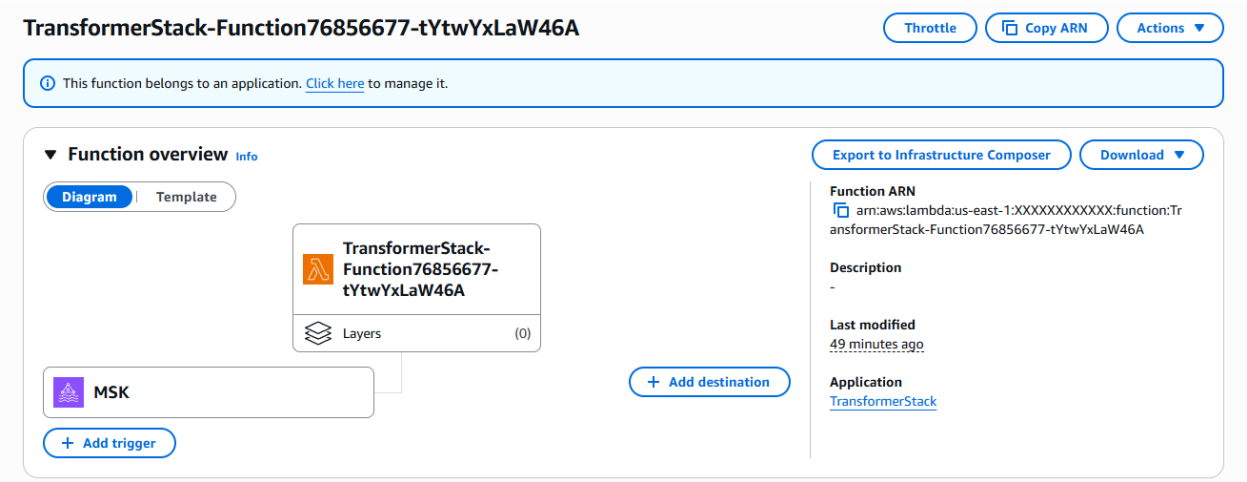


Figure 13. Graph-transformer Lambda function with its Amazon MSK trigger (256 MB, 1-minute timeout).

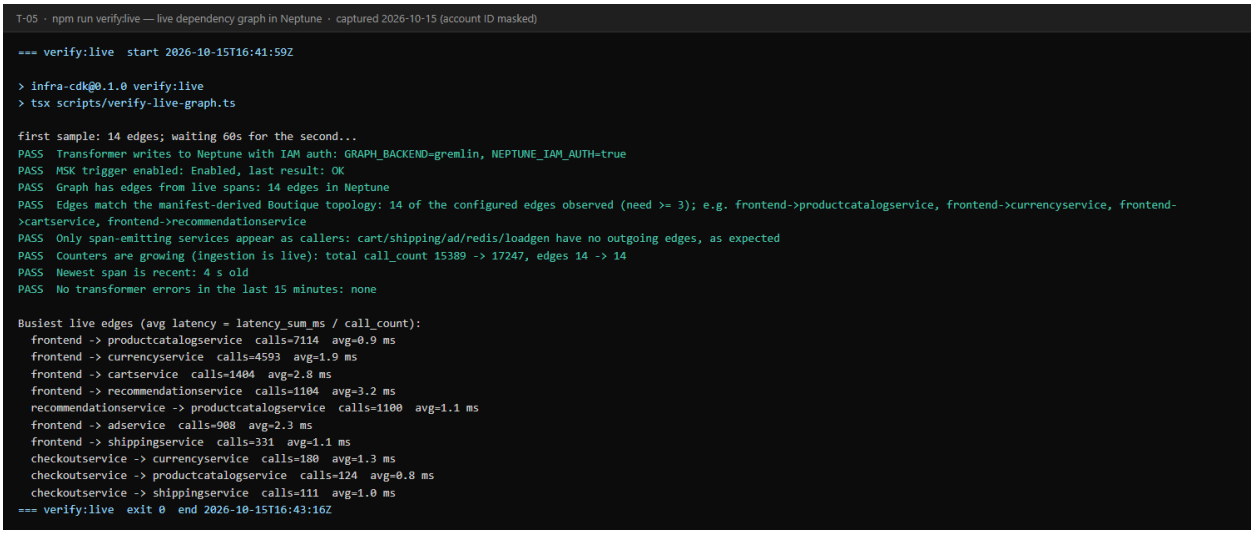


Figure 14. npm run verify:live: fourteen live dependency edges in Neptune, only span-emitting services as callers, counters growing, newest span 4 s old (8/8 PASS).

Spans flow through MSK (Figure 11) into the transformer (Figure 13), which writes the graph in Neptune (Figure 12). In the 15 minutes before the check the transformer processed 308 invocations, 29,391 spans and 14,581 calls without an error; the graph held the fourteen dependency edges of the instrumented services, and its counters grew from 15,389 to 17,247 calls during the check (Figure 14).

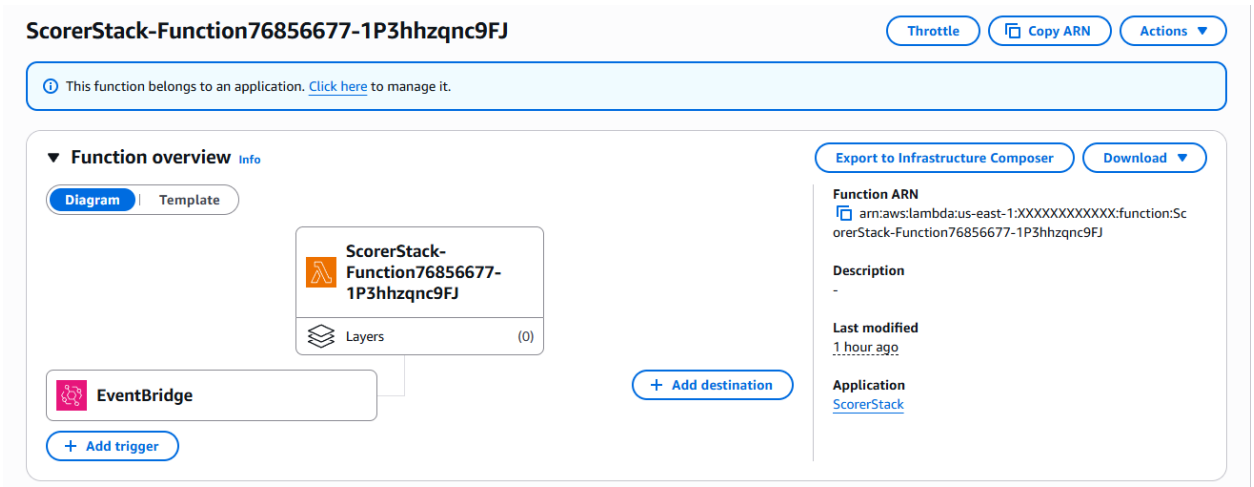


Figure 15. Scorer Lambda function with its Amazon EventBridge schedule (512 MB).

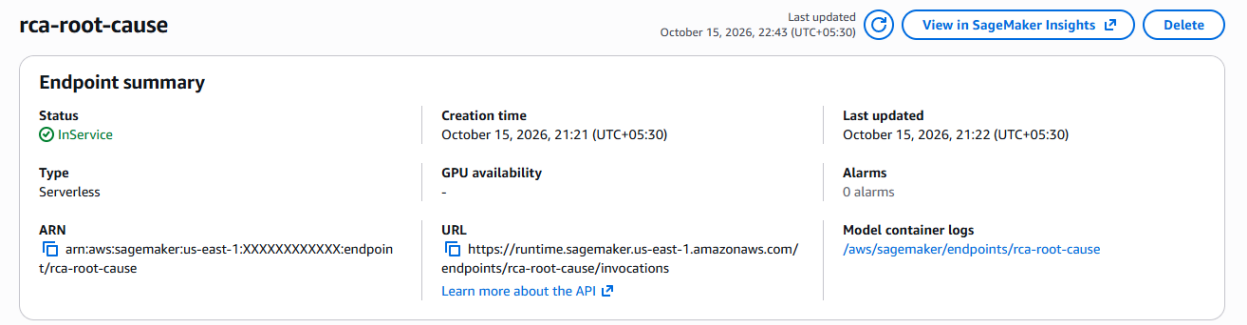


Figure 16. Amazon SageMaker: serverless endpoint rca-root-cause InService.

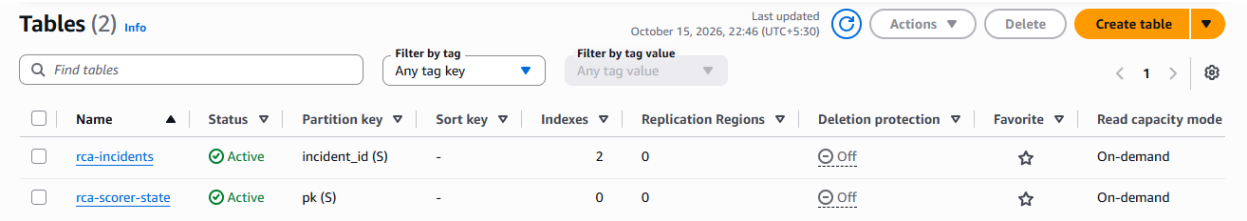


Figure 17. Amazon DynamoDB: the incident table rca-incidents and the scorer state table rca-scorer-state.

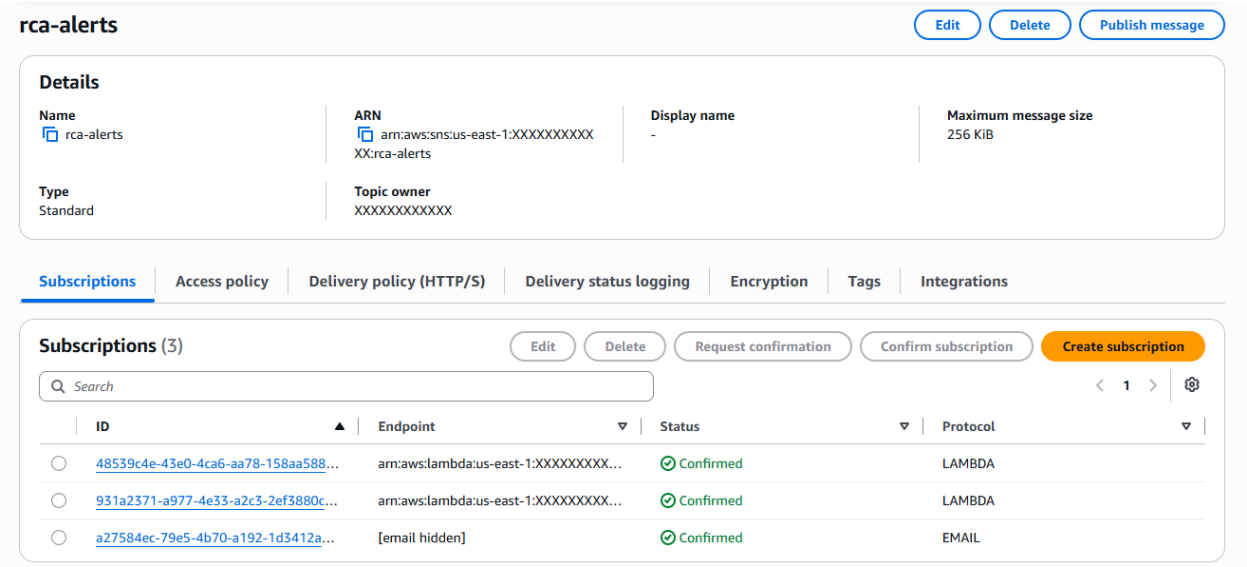


Figure 18. Amazon SNS topic rca-alerts with three confirmed subscriptions: the Slack and PagerDuty notifier functions and one e-mail address (masked).

EventBridge triggers the scorer every minute (Figure 15); the model is served from a serverless endpoint (Figure 16); incidents and scorer state live in DynamoDB (Figure 17); and the alert topic has three confirmed subscribers (Figure 18).

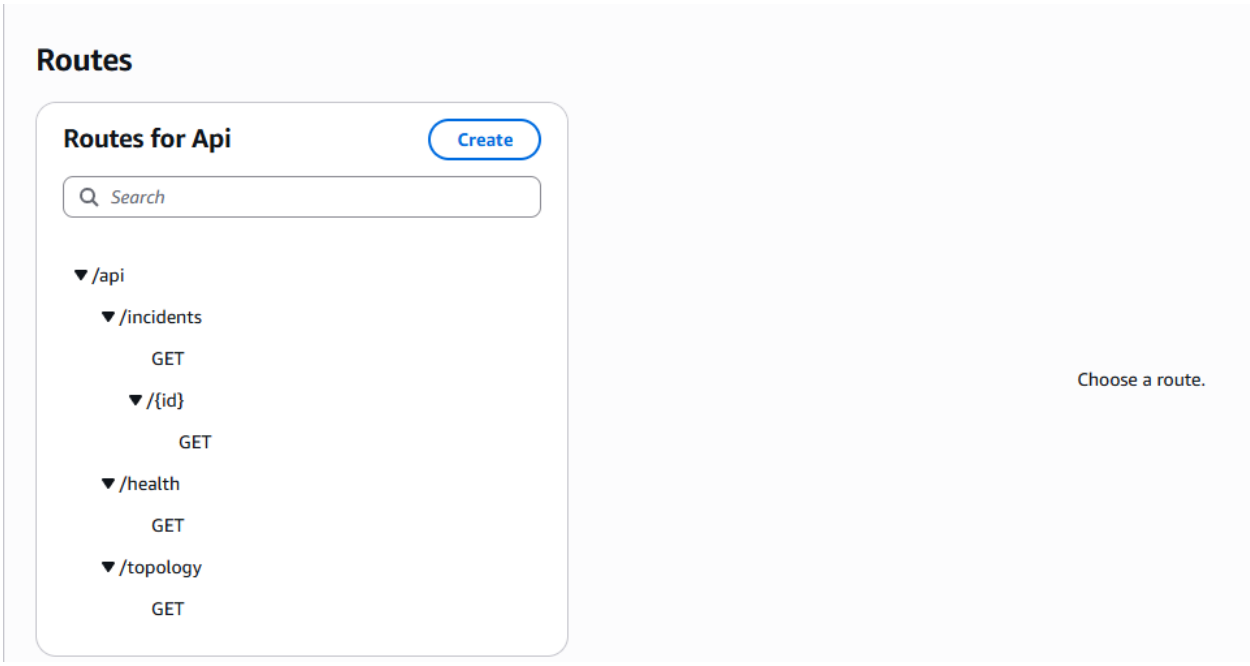


Figure 19. Amazon API Gateway HTTP API: the four read-only GET routes of the dashboard API.

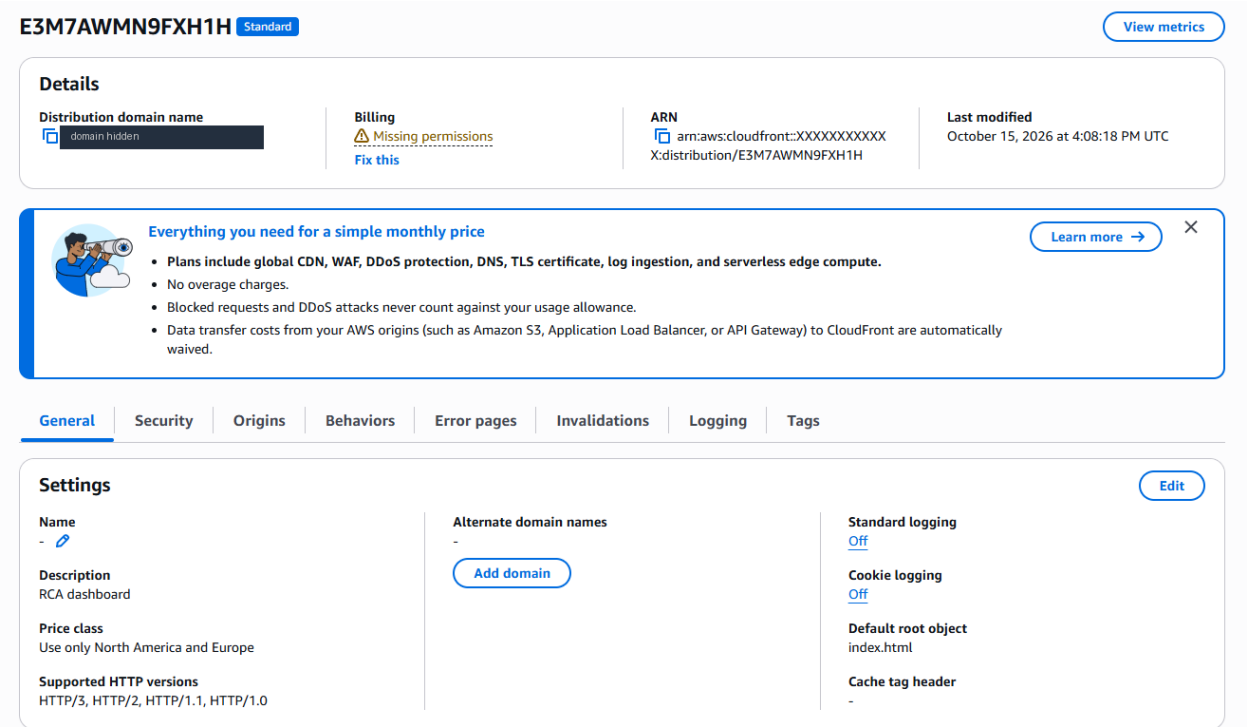


Figure 20. Amazon CloudFront distribution serving the dashboard (domain masked).

The dashboard API exposes four read-only routes (Figure 19) behind the CloudFront distribution (Figure 20).

9.3 GitHub Repository

The repository is public at https://github.com/naveenkumara2024-gif/Intelligent_Microservice_Dependency_Cloud_Project_2026- (Figure 21). Its name differs from the guideline form ProjectName_Cloud_Project_2026 by a trailing hyphen and from the name planned in Phase I (ShoppingPlatform_MicroserviceKG_Cloud_Project_2026). [TO CONFIRM: whether to rename it before submission.]

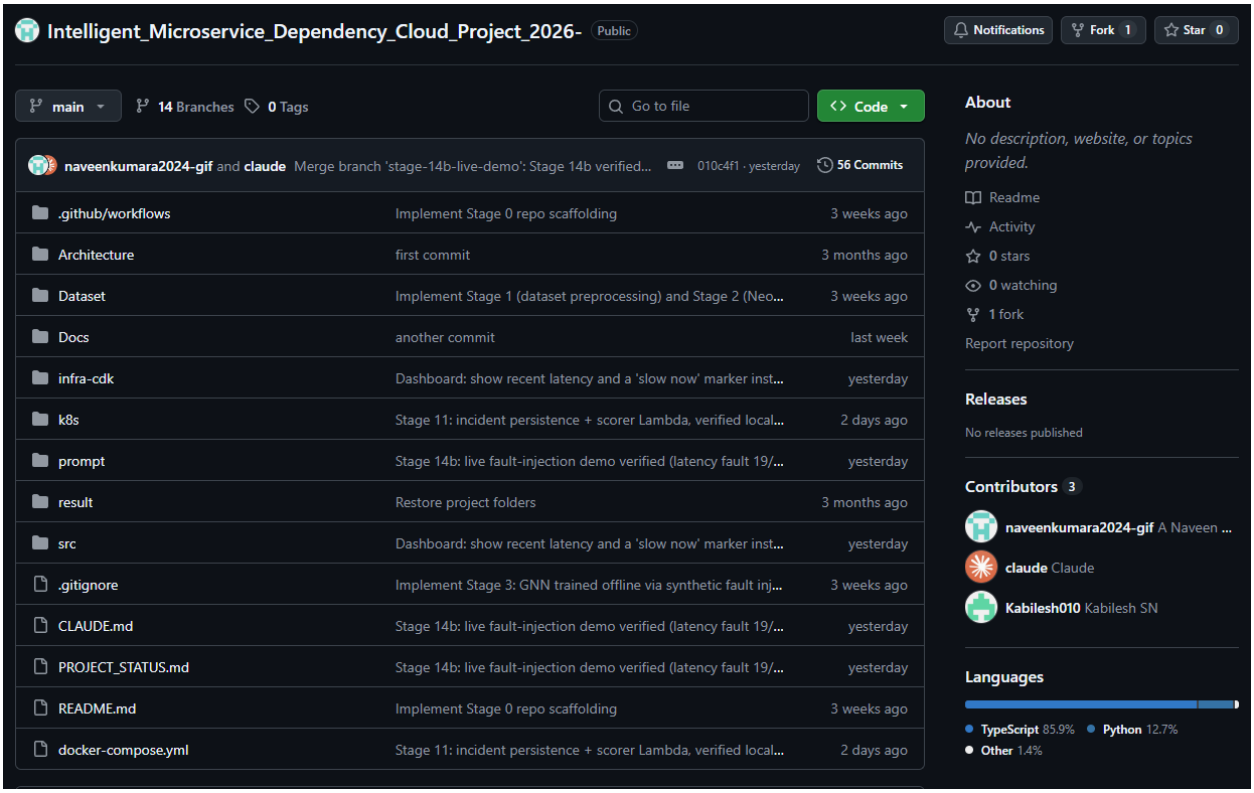


Figure 21. Public GitHub repository: main branch, 56 commits on main, 14 branches, three contributors.

The main folders are:

- README.md, CLAUDE.md, PROJECT_STATUS.md, .gitignore, docker-compose.yml (local Neo4j, Gremlin Server, DynamoDB Local)
- Docs/ – Phase-I report, literature survey, novelty summary, model explanation, report template and this report's build scripts
- Architecture/ – Phase-I architecture diagrams (the as-built diagrams are in Docs/report-build/figures)

- Dataset/ – preprocess_traces.py and load_to_neo4j.py; raw and processed data are gitignored and regenerable
- infra-cdk/ – AWS CDK app (eleven stacks), verification and operations scripts, infrastructure tests
- src/ – collector configuration, Lambda functions (transformer, scorer, alerts, API, probe), ML engine (training, serving, tests) and the web dashboard
- k8s/ – Online Boutique manifest with tracing, Kafka collector overlay, topic-creation job
- prompt/ – the plan and as-built record of every stage; result/ – evidence captured from the live system

10. Testing, Results and Observations

10.1 Test Plan and Test Cases

Test ID	Scenario	Input / setup	Expected result	Observed result	Pass/Fail
T01	Network isolation	verify:vpc after deploying the VPC (6 Oct 2026)	Six subnets in three tiers; no default route in private-data; endpoints available	As expected; destroyed afterwards	Pass
T02	Cluster and workloads	verify:eks (15 Oct 2026)	Nodes Ready in two zones; Boutique pods Running; collector 2/2; spans from 7 services	2/2 nodes (1a, 1b); 12 pods; collector 2/2; 7 services (4/4)	Pass
T03	Streaming	verify:msk (15 Oct 2026)	Cluster active; three topics; spans readable	ACTIVE, 2 brokers; 200 messages from all 7 emitters (5/5)	Pass
T04	Graph transformer	verify:transformer (15 Oct 2026)	Trigger enabled; spans processed without errors	308 invocations, 29,391 spans, 0 errors (5/5)	Pass
T05	Live graph	verify:live (15 Oct 2026)	Edges from live spans; only span-emitting callers; counters growing	14 edges; counters 15,389 -> 17,247; newest span 4 s old (8/8)	Pass

Test ID	Scenario	Input / setup	Expected result	Observed result	Pass/Fail
T06	Neptune security and parity	verify:neptune (13 Oct 2026)	Signed requests accepted, unsigned refused; 8 concurrent writers exact; parity with CSV and Neo4j	All checks passed, top-2,000 and full 16,115-edge graphs identical	Pass
T07	Model endpoint	verify:sagemaker (15 Oct 2026)	InService; serverless; injected fault in top 3; parity within 1e-4; malformed input is 4xx	Fault ranked #1 (0.939); max difference 9.9e-10; HTTP 400 (6/6)	Pass
T08	Model accuracy gate	Boutique-shaped graph, 600 synthetic scenarios	Top-1 >= 60 %, top-3 >= 90 %	Top-1 65.5 %, top-3 98.7 %	Pass
T09	Incident store	verify:dynamodb (13 Oct 2026)	Keys, indexes, TTL, optimistic concurrency	13/13	Pass
T10	Incident loop (synthetic slowdown)	verify:incidents (13 Oct 2026)	No incident in a healthy soak; one incident; resolution	0 incidents in 8 min; one incident in 3.2 min (score 0.994); resolved 4.0 min after the slowdown stopped (22 PASS)	Pass
T11	Alert delivery	verify:alerts (13 Oct 2026)	One alert per transition; no alert for in-place updates; DLQ empty	18 PASS, 2 WARN (Slack and PagerDuty credentials not configured)	Pass with warnings
T12	Dashboard through CloudFront	verify:dashboard --topology (15 Oct 2026)	HTTPS with security headers; HTTP redirected; incident lifecycle; live topology	20 PASS, 2 WARN (cache-time check expects 10 s, set to 5 s; no 429 seen)	Pass
T13	Security: direct API access	Raw execute-api URL without, then with a wrong, origin header	HTTP 403 both times	403 and 403	Pass
T14	Security: encryption at rest	Read-only console session opens incident items	Access denied without KMS permission	Denied: kms:Decrypt (Figure 29)	Pass

Test ID	Scenario	Input / setup	Expected result	Observed result	Pass/Fail
T15	Performance: request burst	30 rapid API requests	No server errors	13 Oct: 503s at the account limit of 10 concurrent executions; 15 Oct, after the limit was raised to 1,000: 30 x HTTP 200	Fail, then Pass
T16	End-to-end real latency fault	verify:demo, 300 ms on productcatalogservice (14 and 15 Oct 2026)	One incident within 5 windows; root cause in top 3; one alert each way; DLQ empty	Both runs 19/19; 15 Oct: opened after 1.1 min, ranked #1 (0.950), resolved 3.9 min after clearing	Pass
T17	End-to-end CPU starvation	verify:demo, CPU limit 15m on currencyservice (14 Oct 2026)	Slowdown detected	Starved pod never became Ready; the old pod kept serving, so no slowdown reached traffic	Inconclusive
T18	Deployment automation	npm run up (15 Oct 2026)	All stacks and workloads in under 50 min	Eleven stacks in about 41 min (15:50–16:31 UTC)	Pass
T19	Teardown	npm run down and verify:teardown	Nothing billable left	[TO CONFIRM after npm run down]; the 13-check sweep passed on the empty account before deployment	Pending
T20	Automated test suites	Lambda, dashboard, infrastructure, Python (15 Oct 2026)	All tests pass	647 tests: 289 + 50 + 257 + 51, all passed	Pass

10.2 Results

All results below were measured on AWS in us-east-1 with the configuration of Chapter 7, under the load generated by Online Boutique's own load generator. Model accuracies come from synthetic scenarios; the live results come from real faults injected into running services.

Live fault test (15 October 2026). Figure 22 shows the run minute by minute. After an eight-minute healthy soak with no model calls and no incidents, 300 ms of latency was added to every

product-catalogue request. Latency into the service rose from 0.88 ms to 301.29 ms (x344) while traffic fell from 982 to 619 calls per minute. The first scoring window after the fault implicated productcatalogservice; the second opened the incident, 1.1 minutes after the fault. The model ranked productcatalogservice first with a peak score of 0.950 and recommendationservice second; recommendationservice was reported as the blast radius, since its own latency rose from 3.0 ms to 52.6 ms only because it calls the faulty service, and checkout, frontend and recommendation were listed as at risk. After the fault was cleared, three clean windows resolved the incident 3.9 minutes later. Exactly one "opened" and one "resolved" alert were sent; SNS delivered all three notifications (two functions and e-mail) with none failed, and the dead-letter queue stayed empty. The same test on 14 October also passed 19 of 19 checks (rank #1, score 0.932).

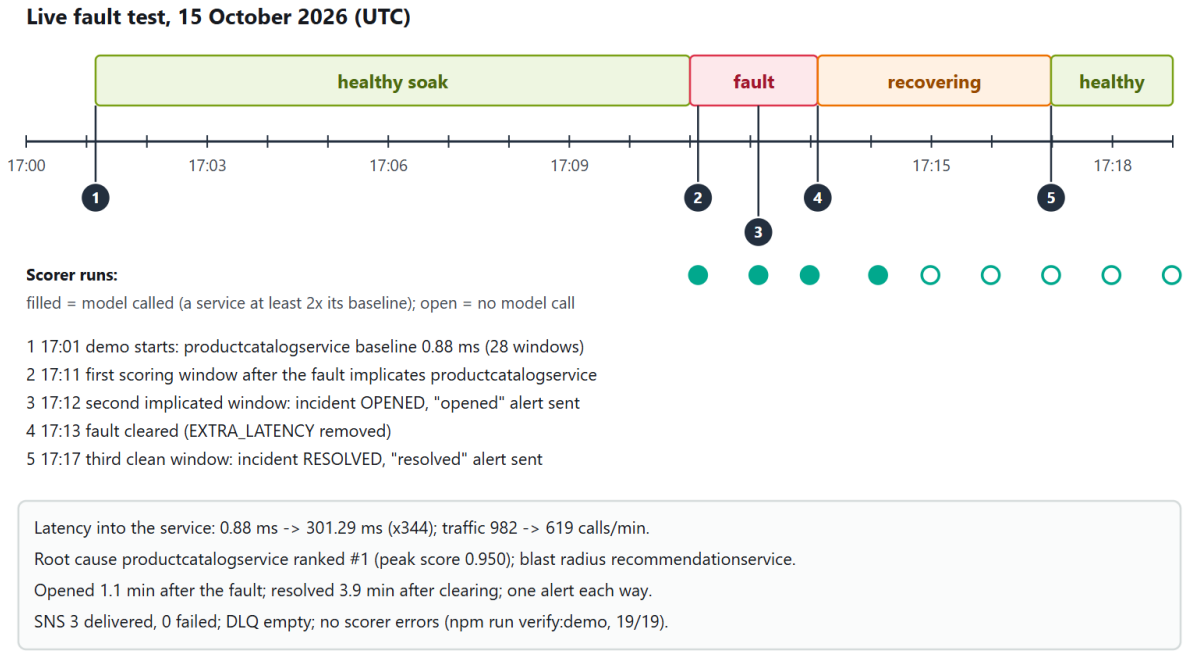


Figure 22. The live fault test of 15 October 2026, minute by minute (UTC): healthy soak, 300 ms fault, incident, clearance and resolution.

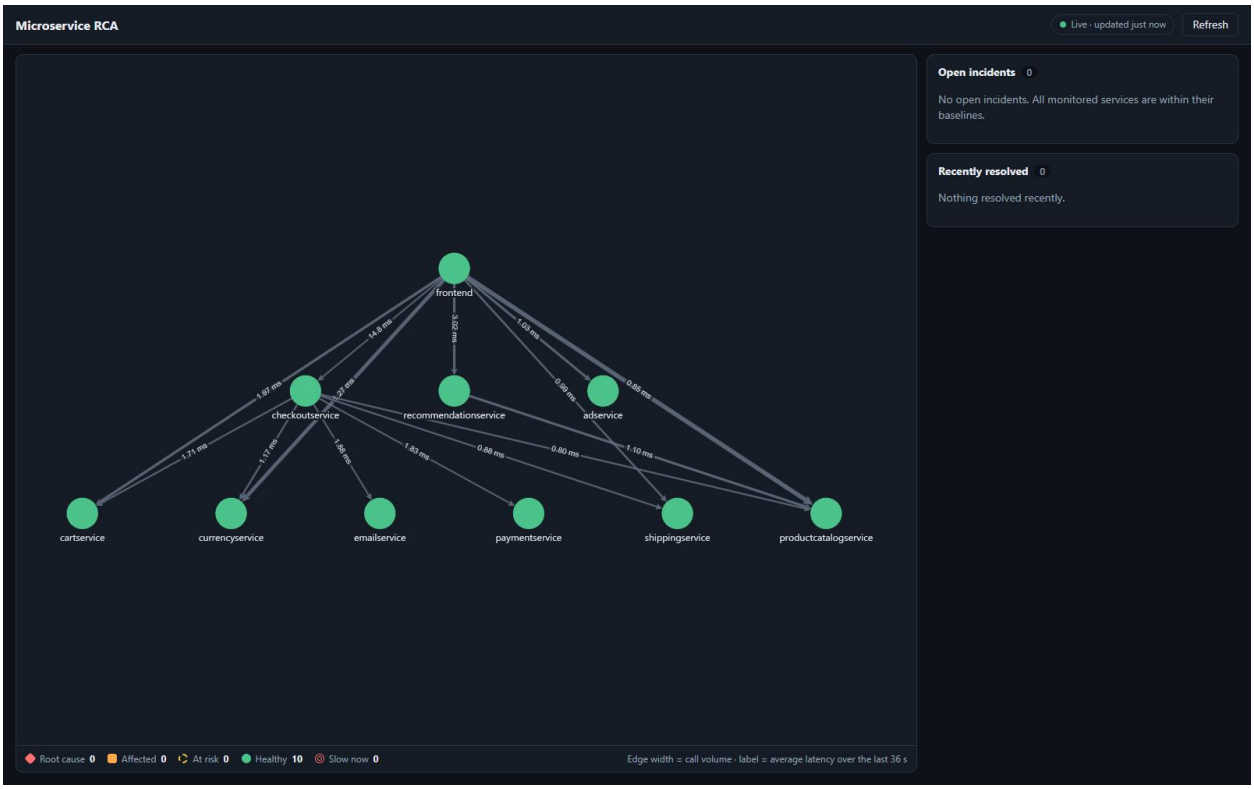


Figure 23. Dashboard before the fault (17:01 UTC): ten healthy services; edge labels are average latency over the last scoring window.



Figure 24. Dashboard during the fault (17:12 UTC): productcatalogservice marked as root cause, recommendationservice as affected, frontend and checkout at risk; the slow edges show about 301 ms.

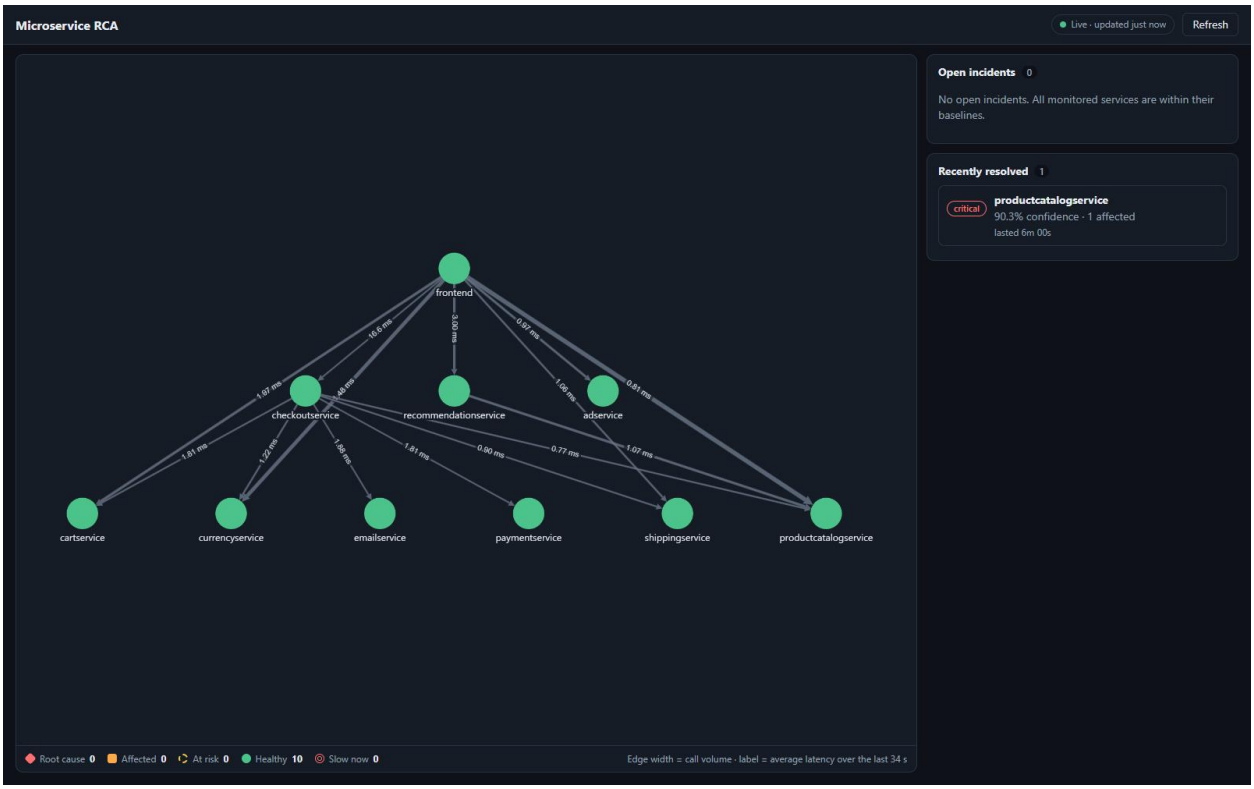


Figure 25. Dashboard after resolution (17:17 UTC): all services healthy again; the incident is listed as resolved after 6 min 00 s.

Figures 23–25 show what an engineer saw on the dashboard before, during and after the fault, and Figure 26 the judged verification output.

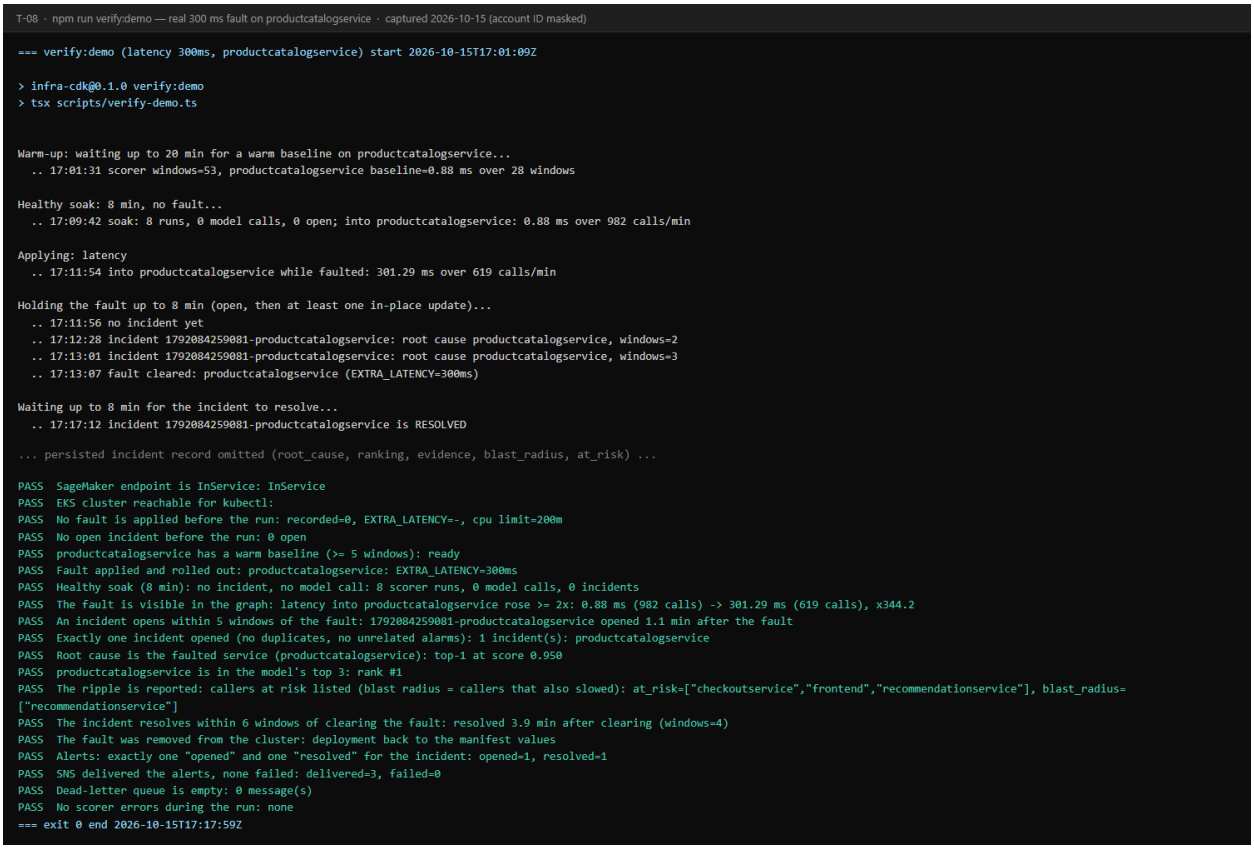


Figure 26. npm run verify:demo on 15 October 2026: the judged run, 19 of 19 checks passed (the persisted incident record is omitted here).

Model accuracy. Figure 27 compares the deployed model with two baselines on a Boutique-shaped graph of 12 services (11 candidates) whose topology was held out of training. In the "proportional" mode, which spreads a fault to callers in proportion to their traffic, the model ranks the true root cause first in 65.5 % of 600 scenarios and in the top 3 in 98.7 %. In the harder "faithful" mode, which follows the full fault-propagation model, it reaches 40.7 % top-1 and 81.7 % top-3. On the Alibaba test scenarios, fine-tuning lowered the original model's accuracy from 62.9 % to 52.0 % top-1 and from 91.8 % to 89.3 % top-3, the price of generalising to small graphs.

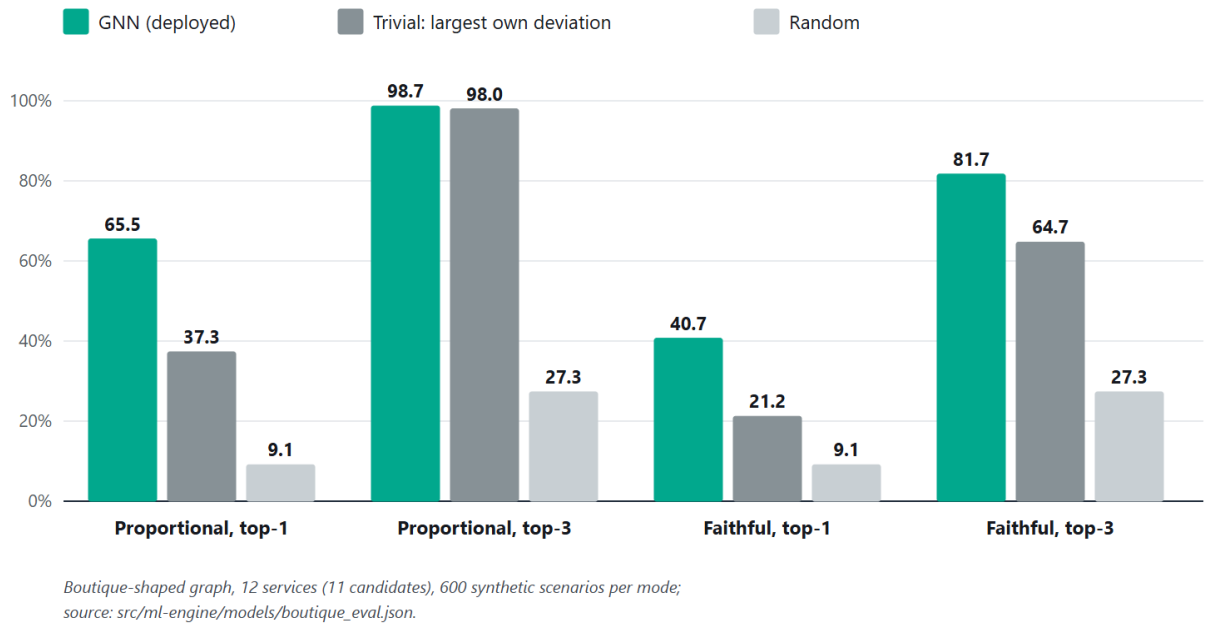


Figure 27. Root-cause ranking accuracy of the deployed model against a trivial and a random baseline on 600 synthetic scenarios per mode.

Operation. In the three-hour window of Figure 28, during which the stack had been running for about 75 minutes, the scorer ran 75 times (once a minute) without an error or throttle; its average duration was 630 ms, rising to about 9 s in the minutes in which it called the model (17.3 s at most). The endpoint answered in 13.7 s cold and 1.9 s warm when first measured in Stage 10, and its scores match local inference to within 9.9e-10. All 647 automated tests pass.

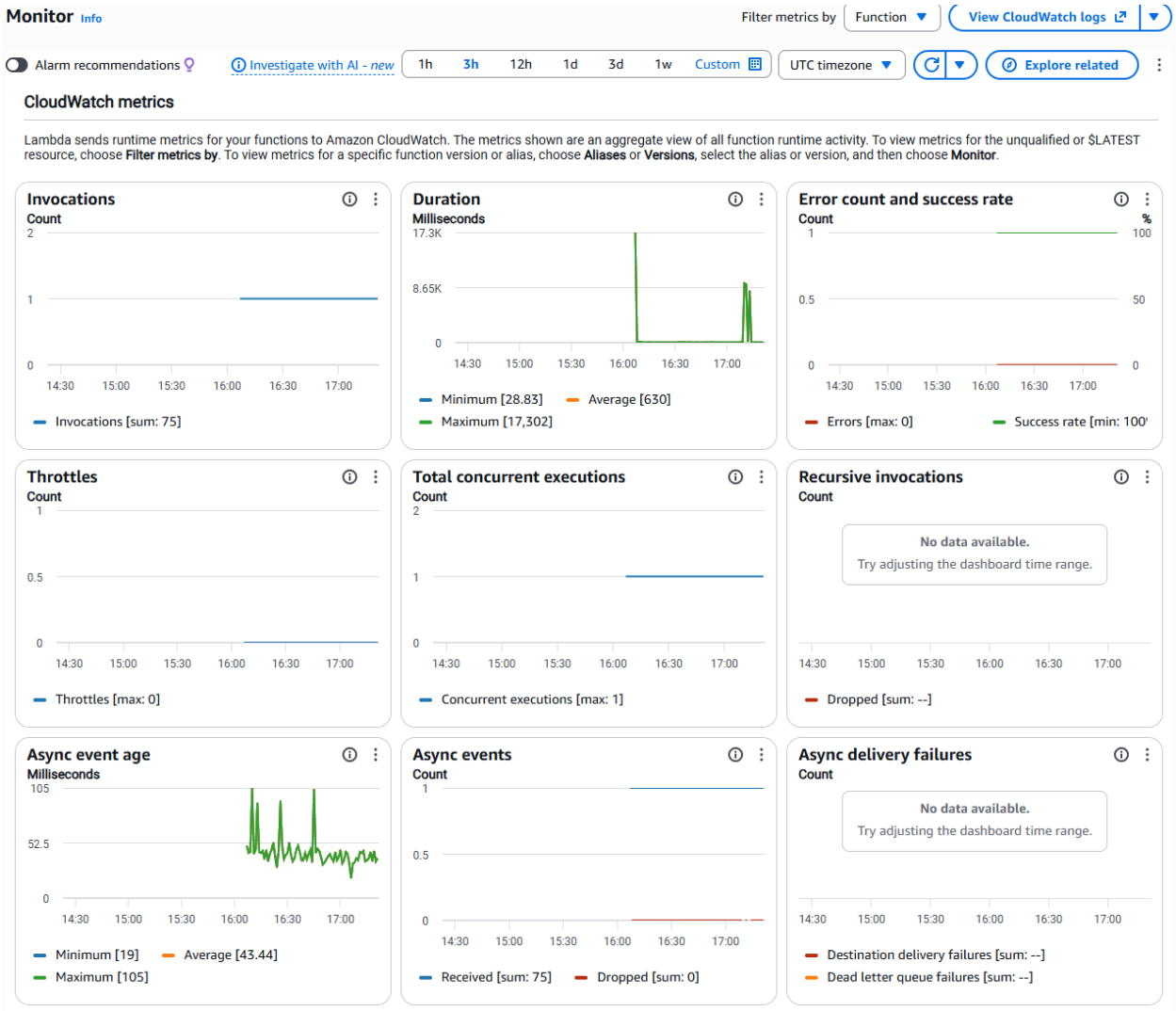


Figure 28. Scorer function metrics over three hours: 75 invocations, no errors, no throttles; duration peaks are the minutes in which the model was called.

Security. Direct calls to the API Gateway URL were refused with HTTP 403 without the origin secret, unsigned requests to Neptune were refused, and a read-only session could not read encrypted incident data (Figure 29).

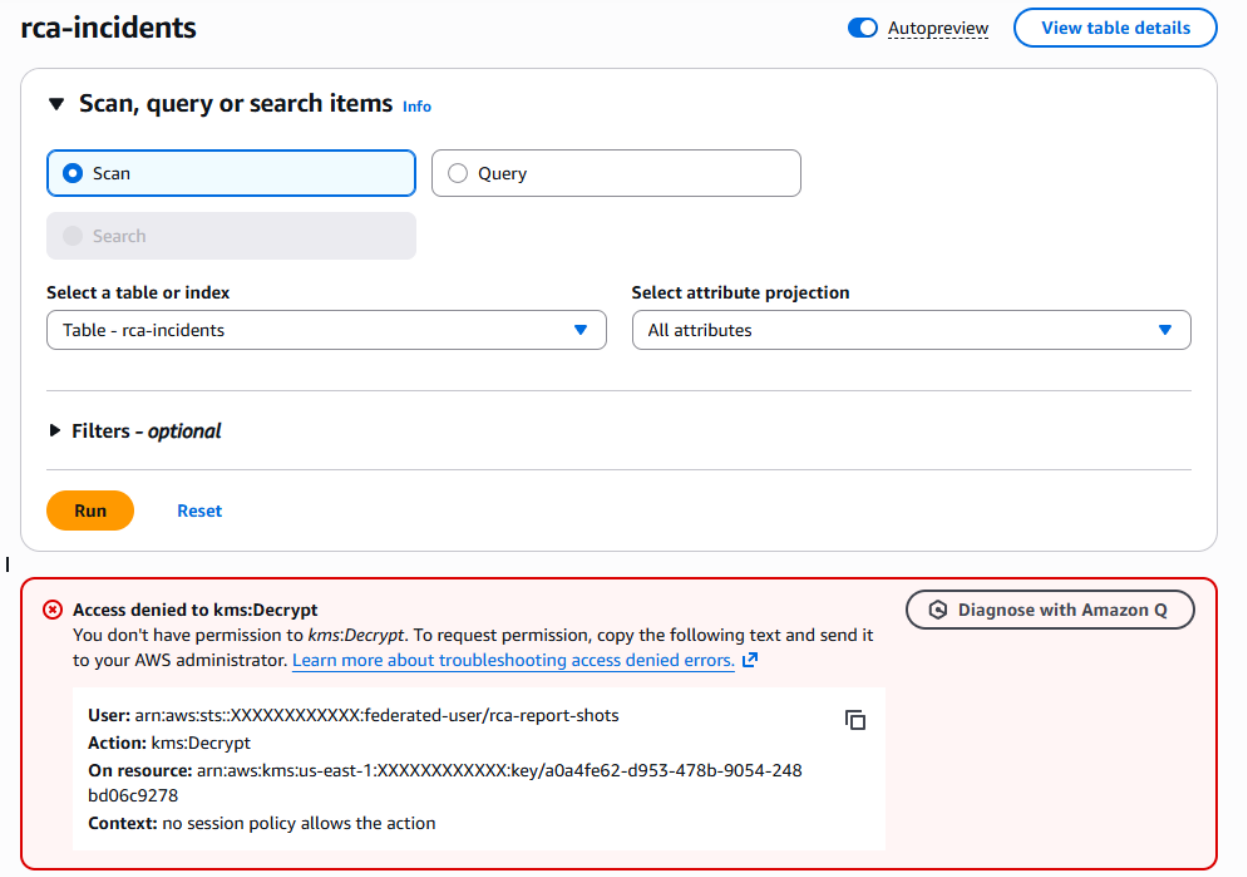


Figure 29. Security check: a session with the AWS ReadOnlyAccess policy cannot read incident items, because they are encrypted with a customer-managed KMS key it may not use.

10.3 Baseline Comparison

Ranking baselines (same scenarios, same metrics). The trivial baseline names the service with the largest own latency deviation, the obvious rule an engineer would apply to a dashboard. On the proportional scenarios it reaches 37.3 % top-1 against the model's 65.5 %; on the faithful scenarios 21.2 % against 40.7 % (Figure 27). The model's advantage is largest for top-1, which is the alert's single named root cause; for proportional top-3 the trivial rule is almost as good (98.0 % against 98.7 %), because the true root cause is usually among the three slowest services. Random choice among 11 candidates gives 9.1 % and 27.3 %.

In the live test the alerting behaviour can be compared with per-service threshold alerting under the same fault: two services crossed the 2x threshold (productcatalogservice and recommendationservice), so per-service alerting would have raised at least two alerts, while the

system raised one alert that named the correct origin and listed the other service as its blast radius.

Graph store parity. The Neptune graph was compared with the Stage-2 Neo4j prototype and with the preprocessed CSV files: both the top-2,000-edge subset and the full 16,115-edge graph were identical. Results from the literature (for example P4's 93.5 % top-3 or P12's 98 % mean average precision) use different datasets and metrics and are therefore not directly comparable with these figures.

10.4 Results Summary

- Objective 1 (continuous graph): achieved; verify:live 8/8 with 14 live edges and growing counters (Figure 14).
- Objective 2 (managed knowledge graph): achieved; parity with Neo4j and the CSV files on 2,000 and 16,115 edges; unsigned requests refused (T06).
- Objective 3 (root-cause ranking): achieved on synthetic Boutique scenarios (65.5 % top-1, 98.7 % top-3); endpoint parity 9.9e-10 (T07, T08).
- Objective 4 (incidents and alerts): achieved for single-service latency faults in two live runs; Slack and PagerDuty delivery is verified only up to their functions, because real credentials have not been configured (T11, T16).
- Objective 5 (visualisation): achieved; the dashboard showed the slowdown and the incident within one scoring window (Figures 24, T12).
- Objective 6 (secure, cost-controlled operation): deployment achieved in about 41 minutes with private data subnets, least privilege and encryption (T13, T14, T18); teardown [TO CONFIRM after npm run down].

11. Discussion, Limitations and Risk Analysis

11.1 Discussion

The live tests show that the system does what Chapter 1 asked for in the case it was built for: a single real fault was turned into one alert that named the faulty service, within about a minute of the slowdown and without false incidents during healthy traffic. The 2x gate kept model calls

(and serverless cold starts) to the minutes that mattered, and the incident rules (two windows to open, three to resolve) avoided alerts on single noisy minutes at the cost of one to two extra minutes of delay.

The model's value lies in top-1 accuracy, where it roughly doubles the trivial rule; that matters because an alert names one service. The building of the system mattered as much as the model: nine defects appeared only on AWS (for example, gRPC client spans name the callee by RPC method and cluster IP, Neptune treats property overwrites as deletions for authorisation, and Node 22's built-in WebSocket dropped the SigV4 headers), and each was found by a verification script and fixed with a regression test. This supports the project's framing that the contribution is the dependable system around graph-based analysis.

11.2 Limitations

- Labels are synthetic: the Alibaba trace has no failure field, so the model never saw a labelled real fault, and its accuracy figures come from simulated scenarios.
- Live evidence covers one kind of fault (a single-service latency fault, two runs), not a statistical sample, multiple simultaneous faults or error-rate faults.
- The model was fine-tuned on small graphs, which lowered its accuracy on the large Alibaba graph (52.0 % top-1).
- Cart, shipping, ad, redis-cart and the load generator emit no spans, so they have no outgoing edges and cannot be blamed for their own downstream calls.
- Thin traffic: checkout, payment and email often stayed below 20 calls per minute, so they rarely acquired a baseline; the 1 ms latency floor makes the gate insensitive to changes below about 2 ms.
- A CPU-starvation fault at a 15-millicore limit never reached traffic (the pod failed its health checks); milder limits are untested.
- Real Slack and PagerDuty delivery and SSM secret decryption have not been exercised with real credentials.
- The dashboard's origin-secret guard is not user authentication, and the default CloudFront certificate cannot enforce a minimum TLS version.
- The account cannot create Network Load Balancers, so the storefront stays private; the system runs in one Region with a single Neptune writer by default.

11.3 Risks and Mitigation

Risk	Likelihood / impact	Mitigation	Residual risk
Unexpected AWS cost	Medium / high (about USD 0.60 per hour while up)	Budget alert checked before deployment; npm run down after every session; verify:teardown sweep; serverless and on-demand services where possible	Low
Credential exposure / misconfiguration	Low / high	No keys in code or Git (.gitignore checked); SSM SecureString and Secrets Manager; least-privilege roles; private data tier; screenshots masked	Low
Dataset quality / availability	Medium / medium	Schema verified against the real file; invalid rows filtered and counted; RPC duplicates removed; synthetic labels documented as such	Medium (no real fault labels)
Account quotas and restrictions	High / medium (Free-Tier plan blocked MSK and Neptune; Lambda concurrency 10; CloudFront needed verification; NLBs refused)	Plan upgraded; quota raised to 1,000; AWS Support case; architecture avoids load balancers	Low
Model domain shift	Medium / high	Fine-tuning on small graphs with a held-out Boutique gate; top-3 ranking and blast radius shown to engineers	Medium
Missing telemetry	High / medium	Documented: services without spans cannot be blamed; future instrumentation	Medium
Teardown leaves resources	Low / medium	Kubernetes load balancers removed first; dependency guard; parallel destroy; 13-check verify:teardown	Low

12. Team Contribution and GitHub Management

The table records each member's individual work. Entries supported by Git history are filled in; the remaining entries are to be completed by the team. [TO CONFIRM: all contributions.]

Activity	Naveen Kumar (24BIT0443)	Kabilesh SN (24BIT0464)	[TO CONFIRM: Student 3]
Literature survey and assigned paper analysis	[TO CONFIRM: papers]	[TO CONFIRM: papers]	[TO CONFIRM: papers]
Research gap and novelty	[TO CONFIRM]	[TO CONFIRM]	[TO CONFIRM]
Frontend / application	Stage-13 dashboard (React, Cytoscape.js) and API	Early frontend work (Aug 2026): implementation merged in PR #3, then a React/TypeScript prototype with dashboard, dependency-graph and services pages on branch frontend	[TO CONFIRM]
Backend / APIs / database	Graph transformer, scorer, DynamoDB incident store, dashboard API	[TO CONFIRM]	[TO CONFIRM]
AWS services and security	CDK stacks (Stages 4–15), VPC, EKS, MSK, Neptune, SageMaker, SNS, CloudFront; IAM and KMS design	[TO CONFIRM]	[TO CONFIRM]
Dataset / AI-ML (if applicable)	Alibaba preprocessing, fault injection, GNN training and fine-tuning, serving model	[TO CONFIRM]	[TO CONFIRM]
Testing and results	Unit, integration and infrastructure tests; live verification and fault tests	[TO CONFIRM]	[TO CONFIRM]
Documentation and presentation	Stage plans and records; this report	[TO CONFIRM]	[TO CONFIRM]

12.1 Branch and Pull Request Workflow

The repository uses main as the integration branch. Early work used team branches (backend, frontend, aws-cloud-mlmodel) merged through pull requests #1–#3 (August 2026). From Stage 3 onwards each stage was built on its own branch (stage-3-gnn-training to stage-14b-live-demo), checked against its checkpoint, and merged into main: Stages 3, 4, 6 and 7 (with the Stage-3 follow-up and Stage-6 fixes) through pull requests #4–#9 on GitHub, Stage 5 and later stages through direct commits and no-fast-forward merges that keep each stage's history visible. Every stage's plan and as-built record is kept in prompt/. No release tag has been created yet. [TO CONFIRM: tag the submitted version, for example v1.0.]

Development was assisted by an AI coding assistant (Claude Code): 32 of the 61 commits on all branches carry its co-author line. Each stage's plan was approved by the team before it was built,

and all AWS deployments were run by the team. [TO CONFIRM: wording required by the course's policy on AI assistance.]

12.2 Contribution Evidence

- Naveen Kumar – GitHub naveenkumara2024-gif; 53 commits across all branches (repository set-up, Stages 0–15, merges of pull requests #1–#9).
- Kabilesh SN – GitHub Kabilesh010; 8 commits on the backend and frontend branches (18–23 August 2026): an early frontend implementation, merged through pull request #3 on 18 August, and a React/TypeScript frontend prototype with dashboard, dependency-graph and services pages on the frontend branch (not merged into main; the Stage-13 dashboard later replaced it).
- [TO CONFIRM: Student 3 – GitHub username, commits, pull requests and reviews.]
- Commit counts are taken from git shortlog on 15 October 2026 and are reported as they are, including where they fall below the guideline of 20–30 commits per student. [TO CONFIRM: code-review participation.]

13. Conclusion and Future Enhancements

13.1 Conclusion

Cascading failures in microservice shops produce many symptoms for one cause, and per-service monitoring leaves engineers to find the origin by hand. This project built a streaming, cloud-deployed framework that keeps a live, latency-weighted dependency graph of the shop in Amazon Neptune, scores it every minute against learned baselines, and asks a graph neural network on Amazon SageMaker to rank the root cause only when a service is clearly slower than usual. Incidents are stored in DynamoDB, alerted once through SNS and shown on a CloudFront dashboard; the whole system is eleven CDK stacks that deploy in about 41 minutes and can be removed with one command.

Every stage was verified on AWS. In two live tests a real 300 ms fault produced exactly one incident that named the faulty service first, opened about a minute after the fault and resolved about four minutes after it was removed, with no false incidents during healthy traffic. The

model's accuracy (65.5 % top-1, 98.7 % top-3 on held-out Boutique-shaped scenarios) comes from synthetic faults, and the live evidence covers single-service latency faults. Objectives 1–5 were met within these limits; Objective 6 was met for deployment, with teardown to be confirmed.

13.2 Future Enhancements

- Run repeated live experiments with several simultaneous faults, error-rate faults and cascades, and report results as statistics rather than single runs.
- Instrument the silent services (cart, shipping, ad) and add temporal features so that the model sees trends across windows.
- Configure real Slack and PagerDuty credentials and verify their delivery and SSM decryption end to end.
- Add Amazon Cognito sign-in to the dashboard and a custom domain with an enforced minimum TLS version.
- Package each Lambda function's bundle separately so that a change to one function redeploys only that function.
- Expose the storefront through an Application Load Balancer, enable the Neptune read replica, and evaluate under production-scale load and in a second Region.

References

[1] Gu, Y., Zhang, J., & Du, Y. (2026). Causal graph-enhanced large language models for automated fault diagnosis and intelligent operation and maintenance in distributed computing systems. Electronics, 15(11), 2359. https://doi.org/10.3390/electronics15112359

[2] Zhang, J., & Yang, H. (2026). CPU-only spatiotemporal anomaly detection in microservice systems via dynamic graph neural networks and LSTM. Symmetry, 18(1), 87. https://doi.org/10.3390/sym18010087

[3] Phan, L.-A., Wang, M., Wu, G., Wang, D., Chen, L., & Li, J. (2025). CrossTrace: Efficient cross-thread and cross-service span correlation in distributed tracing for microservices (arXiv:2508.11342) [Preprint]. arXiv. https://arxiv.org/abs/2508.11342

[4] Wang, T., Qi, G., & Wu, T. (2024). KGroot: Enhancing root cause analysis through knowledge graphs and graph convolutional neural networks (arXiv:2402.13264) [Preprint]. arXiv. https://arxiv.org/abs/2402.13264

[5] Pham, L., Ha, H., Zhang, X., & Zhang, H. (2026). TORAI: Multi-source root cause analysis for blind spots in microservice service call graph (arXiv:2604.13522) [Preprint; accepted at FSE 2026]. arXiv. https://arxiv.org/abs/2604.13522

[6] Zheng, L., Chen, Z., & Chen, H. (2024). Online multi-modal root cause identification in microservice systems (arXiv:2410.10021) [Preprint; accepted at IEEE BigData 2025]. arXiv. https://arxiv.org/abs/2410.10021

[7] Madabhushi, S., Vyas, P., Vaidyanathan, S., Kurup, M., Nash, E., & Silyutin, Y. (2026). From load tests to live streams: Graph embedding-based anomaly detection in microservice architectures (arXiv:2604.06448) [Preprint; accepted at FSE 2026 Industry Track]. arXiv. https://arxiv.org/abs/2604.06448

[8] Zhang, L., Jia, T., Zhai, Y., Pan, L., Duan, C., He, M., Xiao, P., & Li, Y. (2026). Hypothesize-then-verify: Speculative root cause analysis for microservices with pathwise parallelism (arXiv:2601.02736) [Preprint; accepted at ICSE-NIER 2026]. arXiv. https://arxiv.org/abs/2601.02736

[9] Zhang, L., Jia, T., Wang, K., Duan, C., He, M., Wang, R., Peng, X., Wang, M., Zhang, G., Chen, R., & Li, Y. (2026). Towards in-depth root cause localization for microservices with multi-agent recursion-of-thought (arXiv:2605.14866) [Preprint]. arXiv. https://arxiv.org/abs/2605.14866

[10] Colarusso, C., De Caro, A., Falco, I., Goglia, L., & Zimeo, E. (2024). A distributed tracing pipeline for improving locality awareness of microservices applications. Software: Practice and Experience, 54(6), 1118–1140. https://doi.org/10.1002/spe.3317

[11] Sui, Y., Zhang, Y., Sun, J., Xu, T., Zhang, S., Li, Z., Sun, Y., Guo, F., Shen, J., Zhang, Y., Pei, D., Yang, X., & Yu, L. (2023). LogKG: Log failure diagnosis through knowledge graph. IEEE Transactions on Services Computing, 16(5), 3493–3507. https://doi.org/10.1109/TSC.2023.3293890

[12] Behera, A., Panigrahi, C. R., Behera, S., Patel, R., & Bera, S. (2023). trACE – Anomaly correlation engine for tracing the root cause on a cloud based microservice architecture. Computación y Sistemas, 27(3), 791–800. https://doi.org/10.13053/cys-27-3-4498

[13] Dilini, N., Sun, N., Miao, Y., & Moustafa, N. (2026). A comprehensive review on graph-based anomaly detection: Approaches for intrusion detection. Applied Sciences, 16(4), 1906. https://doi.org/10.3390/app16041906

[14] Khan, K., Rahman, Z., Khan, A., Roy, A., & Kabir, J. (2024). Microservices-based system design for ensuring high availability and system reliability. The Eastasouth Journal of Information System and Computer Science, 2(02), 265–274. https://doi.org/10.58812/esiscs.v2i02.1077

[15] Wang, T., & Qi, G. (2024). A comprehensive survey on root cause analysis in (micro) services: Methodologies, challenges, and trends (arXiv:2408.00803) [Preprint]. arXiv. https://arxiv.org/abs/2408.00803

[16] Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J., & Xu, C. (2021). Characterizing microservice dependency and performance: Alibaba trace analysis. In Proceedings of the ACM Symposium on Cloud Computing (pp. 412–426). ACM. https://doi.org/10.1145/3472883.3487003

[17] Brody, S., Alon, U., & Yahav, E. (2022). How attentive are graph attention networks? International Conference on Learning Representations (ICLR 2022). https://arxiv.org/abs/2105.14491

[18] Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. Advances in Neural Information Processing Systems 30 (NIPS 2017). https://arxiv.org/abs/1706.02216

[19] Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. ICLR 2019 Workshop on Representation Learning on Graphs and Manifolds. https://arxiv.org/abs/1903.02428

[20] Google Cloud Platform. (2026). Online Boutique (Version 0.10.7) [Computer software]. GitHub. https://github.com/GoogleCloudPlatform/microservices-demo

[21] OpenTelemetry Authors. (n.d.). OpenTelemetry documentation. Retrieved October 15, 2026, from https://opentelemetry.io/docs/

[22] Amazon Web Services. (n.d.). What is Amazon EKS? Amazon EKS User Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html

[23] Amazon Web Services. (n.d.). What is Amazon MSK? Amazon Managed Streaming for Apache Kafka Developer Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/msk/latest/developerguide/what-is-msk.html

[24] Amazon Web Services. (n.d.). Using Amazon MSK with AWS Lambda. AWS Lambda Developer Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/lambda/latest/dg/with-msk.html

[25] Amazon Web Services. (n.d.). Authenticating your Amazon Neptune database with AWS Identity and Access Management. Amazon Neptune User Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/neptune/latest/userguide/iam-auth.html

[26] Amazon Web Services. (n.d.). Deploy models with Amazon SageMaker Serverless Inference. Amazon SageMaker Developer Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/sagemaker/latest/dg/serverless-endpoints.html

[27] Amazon Web Services. (n.d.). Using time to live (TTL) in DynamoDB. Amazon DynamoDB Developer Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html

[28] Amazon Web Services. (n.d.). Restrict access to an Amazon S3 origin. Amazon CloudFront Developer Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html

[29] Amazon Web Services. (n.d.). What is the AWS CDK? AWS Cloud Development Kit (AWS CDK) v2 Developer Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/cdk/v2/guide/home.html

[30] Amazon Web Services. (n.d.). Using the AWS Price List Query API. AWS Billing User Guide. Retrieved October 15, 2026, from https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/using-price-list-query-api.html

Appendix A – Project Timeline

Phase / task	Start	End	Owner(s)	Status / evidence
Literature survey	Jul 2026	Aug 2026	[TO CONFIRM]	Complete; Phase-I report; citations re-verified Oct 2026
Research gap and novelty	Jul 2026	Aug 2026	[TO CONFIRM]	Complete; Phase-I novelty summary; revised framing (Chapter 4)
Architecture design	30 Jul 2026	30 Jul 2026	[TO CONFIRM]	Phase-I diagrams; as-built diagrams Oct 2026 (Figures 1, 2)
Dataset, local graph and GNN (Stages 0–3)	21 Sep 2026	22 Sep 2026	Naveen Kumar	Complete; pull requests #4 and #6
Infrastructure as code and networking (Stages 4–5)	21 Sep 2026	6 Oct 2026	Naveen Kumar	Complete; pull request #5; verify:vpc
EKS, MSK, transformer, Neptune (Stages 6–9)	4 Oct 2026	13 Oct 2026	Naveen Kumar	Verified live 12–13 Oct 2026; pull requests #7–#9
SageMaker, incidents, alerting, dashboard (Stages 10–13)	12 Oct 2026	15 Oct 2026	Naveen Kumar	Verified live; dashboard tier 2 on 15 Oct 2026
Implementation and integration: fault demo and operations (Stages 14–15)	13 Oct 2026	15 Oct 2026	Naveen Kumar	verify:demo 19/19 on 14 and 15 Oct 2026; npm run up in about 41 min
Testing and report	15 Oct 2026	[TO CONFIRM]	[TO CONFIRM]	Evidence captured 15 Oct 2026; this report

Appendix B – Review Checklist

☑ Abstract is 200–300 words and describes the problem, challenges, proposed solution, AWS services, and status/results.

☑ 15 research papers are included, with citations and a comparative literature survey.

☐ Each student has completed their own research gap analysis for assigned papers. (Not yet: to be confirmed by each student (Section 2.2).)

☑ 4–6 measurable objectives are listed.

☑ Novelty summary is limited to about one page and is supported by comparison with existing work.

☑ Both required architecture diagrams are included and explained.

☑ Dataset name, source, URL, size, record count, feature count, data type, license, purpose, and preprocessing are documented where applicable.

☑ AWS services, their purpose, status, security considerations, and estimated cost are documented.

☑ Implementation evidence, test cases, actual results, limitations, and future work are included.

☐ GitHub repository, branch workflow, pull requests, commits, and individual contributions are documented. (Not yet: individual contributions of all members to be completed (Chapter 12).)

☑ References use APA style; all borrowed ideas, figures, datasets, and documentation are cited.

☑ No credentials, passwords, tokens, or confidential data appear in the report or repository.

☑ Report follows the specified font, spacing, margin, heading, and page-number formatting.